

Received 17 August 2025, accepted 18 September 2025,
date of publication 30 September 2025, date of current version 17 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3615990

RESEARCH ARTICLE

IntelliUnitGen: A Unit Test Case Generation Framework Based on the Integration of Static Analysis and Prompt Learning

ZHUOYING YANG^{1,4} AND LEI WANG^{1,2,3,4}

¹College of Mathematics and Computer Science, Yan'an University, Yan'an, Shaanxi 716000, China

²School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100081, China

³Hebei Key Laboratory of Big Data Science and Intelligent Technology, Tangshan Research Institute, Beijing Institute of Technology, Tangshan, Hebei 063000, China

⁴Key Laboratory of Industrial Internet and Big Data, China National Light Industry, Beijing 100081, China

Corresponding author: Lei Wang (wanglei0823@foxmail.com)

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 61932004, and in part by the Scientific Research Plan Projects of Shaanxi Education Department under Grant 23JK0724.

ABSTRACT Test case generation is a critical research topic in the field of automated software engineering. In recent years, with the rapid advancement of large language models (LLMs) such as the GPT series, LLaMA series, and DeepSeek series, both domestic and international researchers have attempted to leverage prompt learning capabilities of LLMs for automated test case generation and have achieved promising results. However, most existing approaches rely solely on the language model's ability to respond to natural language prompts, while neglecting systematic analysis of static features within source code. These methods often fail to apply traditional static analysis techniques to extract structural and semantic information from the source code, resulting in incomplete generation, omission of key elements, or semantic misinterpretation. To address these limitations, this paper proposes IntelliUnitGen — an intelligent unit test case generation framework that integrates static analysis with prompt learning. This framework aims to combine the advantages of traditional static analysis techniques with the cutting-edge capabilities of LLMs to enhance the automation level and quality of unit test case generation. IntelliUnitGen synthesizes static code features extracted via static analysis into structured, task-relevant prompts for the language model. It adopts a chain-of-thought prompting strategy to guide the LLM in generating rigorous and comprehensive test cases. The overall workflow of the proposed framework is presented, followed by a detailed introduction to its key techniques and implementation details. An illustrative example is provided to intuitively demonstrate the test case generation process of the framework, and experimental validation is conducted on three open-source projects. Experimental results show that IntelliUnitGen significantly outperforms traditional (non-LLM-based) tools and LLM-only prompt learning approaches in metrics such as coverage, generalizability, and compilability & executability, achieving state-of-the-art (SOTA) performance. This study demonstrates the potential of integrating static code analysis with prompt learning to improve not only unit test case generation but also other source code processing tasks, potentially overcoming existing limitations of LLMs in software engineering applications. This framework demonstrates a practical step forward in addressing long-standing challenges in automated software testing. To facilitate reproducibility and further research, the complete implementation of the IntelliUnitGen framework, alongside the generated test cases and JaCoCo coverage reports for all experimental projects, has been made publicly available at: <https://github.com/yangzy0115/IntelliUnitGen>

INDEX TERMS Software engineering, unit test case generation, prompt learning, static analysis, chain-of-thought prompting, source code processing, large language models, automated software engineering.

The associate editor coordinating the review of this manuscript and approving it for publication was Alba Amato¹.

I. INTRODUCTION

With the rapid development of the software industry, ensuring software quality has become a critical issue [1]. As a

core component of software testing, unit testing aims to verify the correctness of the smallest functional modules in a program to ensure they operate as expected, thereby significantly improving software quality and reliability [2]. The design and execution of test cases are key steps in unit testing, directly affecting the comprehensiveness and effectiveness of the test process. Manually writing test cases is not only time-consuming and labor-intensive but also prone to variability in quality due to the tester's experience level. To address this issue, researchers worldwide have begun exploring automated test case generation techniques to enhance testing efficiency and coverage while reducing human-related limitations. Automatically or semi-automatically generating unit test cases can not only alleviate the workload of developers and testers, but also assist them in gaining deeper insight into program logic and potential risks. This, in turn, improves testing efficiency and system quality, enhances overall controllability in software development and maintenance, and reduces manual costs. As a result, automatic generation of unit test cases has become a prominent research focus in the field of software testing.

Early automated test generation methods primarily focused on static analysis techniques [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]. These approaches systematically analyzed program structures and control flow graphs to generate test inputs that cover specific execution paths, and supplemented them with rule-based assertion design, thereby improving the efficiency of test generation and structural coverage to a certain extent. However, such methods lack contextual awareness and semantic reasoning capabilities, and still face significant challenges when dealing with complex business logic, object dependencies, and large input spaces. In recent years, large language models (LLMs) based on the Transformer architecture [15]. For example, the GPT series [16], [17], [18], [19], [20], [21] and its code-specific large language model Codex [22], the LLaMA series [23], [24], [25] and its code-oriented variant CodeLLaMA [26], the DeepSeek series [27], [28], [29], [30] and its code-focused LLMs, the DeepSeek-Coder series [31], [32], [33], as well as the StarCoder code model [34], have been widely applied to the task of automated test case generation [35], [36], [37], [38], [39], [40], [41], [42], [43], [44], [45]. Researchers typically construct prompts to guide LLMs in analyzing given code segments and automatically generating corresponding test cases. At present, most existing LLM-based test case generation approaches primarily rely on the models' intrinsic reasoning and processing capabilities, using either single-step or step-by-step prompting strategies to drive the generation process. Although LLMs possess powerful capabilities in natural language understanding and generation, and are theoretically able to handle complex program structures and semantic information, they still exhibit certain limitations in code processing compared to traditional static analysis techniques. These limitations include inaccurate understanding of code details, incomplete logical analysis, lack of precise modeling of control and data flows, and insufficient

interpretability of generated results. Such issues may lead to test cases with deficiencies in executability, coverage, and accuracy. Therefore, how to effectively integrate static analysis with prompt learning, construct structured and high-quality prompts, and leverage the complementary strengths of both approaches remains an urgent problem to be addressed.

To this end, we propose an intelligent unit test case generation framework—IntelliUnitGen—which integrates static analysis with prompt learning. This framework fully leverages the advantages of static analysis and the chain-of-thought (CoT) [46] reasoning capabilities of LLMs. By progressively transmitting static structural information and constructing a hierarchical prompt chain, IntelliUnitGen enables LLMs to incrementally abstract and comprehend the structural relationships and behavioral semantics of the target code, thereby facilitating the generation of high-quality, high-coverage, and logically sound test cases. IntelliUnitGen, which integrates code preprocessing, static structure extraction, multi-turn prompt construction, and post-processing mechanisms, combines preprocessing techniques, static analysis, CoT reasoning, LLM Application Programming Interface (API) invocation, and bug-fixing strategies into a unified and intelligent framework for automated unit test generation. We first elaborate on the design philosophy and overall workflow of the framework, followed by a description of the functional roles of each stage. Then, we investigate the key techniques and implementation details across seven stages: preprocessing, static analysis, prompt synthesis, contextual learning, test case generation, post-processing, and test execution with error correction. Subsequently, we present an illustrative example—a discount calculator based on purchase amount—to demonstrate the end-to-end process of the framework in a concrete scenario. On this basis, we apply IntelliUnitGen to two widely used open-source datasets: Commons-CLI [47], Commons-CSV [48], and Binance-Connector dataset [49] generating unit test cases and JaCoCo coverage reports for each. We then evaluate and analyze the experimental results from three key perspectives: test coverage, generalizability and compilability & executability. This study not only helps reduce the cost of manual test case construction and improve the automation and coverage of unit testing, but also provides a novel solution for intelligent testing of large-scale software systems. It promotes the development of LLM-based automated testing methods towards higher efficiency and intelligence.

The main innovations and contributions of this paper are as follows:

A. CONSTRUCTING A UNIT TEST GENERATION STRATEGY INTEGRATING STATIC ANALYSIS AND PROMPT LEARNING

This paper proposes a novel unit test case generation strategy that combines static analysis with prompt learning. By fully leveraging the structural precision and semantic depth provided by static analysis, together with the powerful comprehension and code generation capabilities of LLMs, the

proposed approach enables the generation of higher-quality test cases.

B. DEVELOPING A COMPREHENSIVE FRAMEWORK FOR UNIT TEST GENERATION

This paper develops an automated framework named IntelliUnitGen, which integrates static analysis with LLMs, supports step-by-step prompt learning, and incorporates a complete test generation workflow. The framework achieves engineering-level applicability and state-of-the-art (SOTA) performance. It provides a reusable, extensible, and evaluable engineering solution for the automated generation of unit tests for code with complex logic.

C. RELEASING COMPLETE SOURCE CODE AND DATA

This paper publicly releases the complete source code of the IntelliUnitGen framework, the test cases generated for the experimental projects, and the corresponding coverage reports. This release supports result reproducibility and further research, promotes technological exchange and advancement in related fields, and provides a high-quality, reusable experimental benchmark for the task of unit test case generation.

The structure of this paper is organized as follows: Section II reviews the related work; Section III discusses the methodology and overall workflow of the IntelliUnitGen framework; Section IV presents the key techniques and implementation details of IntelliUnitGen; Section V provides an illustrative example; Section VI conducts experiments and analyzes the results; Section VII discusses the research findings; Section VIII concludes the paper; and Section IX outlines future research directions.

II. RELATED WORK

Early methods for unit test case generation primarily relied on static analysis techniques—such as control flow analysis, data flow analysis, and symbolic execution—to perform structural modeling of source code and solve path conditions, thereby generating test data and assertions that satisfy specific coverage criteria. However, these methods often face limitations such as path explosion, poor scalability, and difficulties in covering dynamic behaviors when dealing with complex business logic, inter-method dependencies, and code with dynamic features. LLMs [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], pretrained on large-scale code corpora via self-supervised learning, have demonstrated powerful capabilities in code understanding, generation, and reasoning. These models not only support multiple programming languages but also enable complex code analysis tasks through few-shot learning and in-context reasoning. They offer new paradigms for intelligent source code processing tasks such as design pattern detection [50], code generation [51], unit test case generation [52], and bug repair [53], thereby advancing software engineering toward greater efficiency, automation, and intelligence. As a result,

the automatic generation of unit test cases using LLMs has emerged in recent years as one of the prominent research hotspots in the field of software engineering. Related studies have gradually evolved from initial exploratory cases to systematic approaches, demonstrating significant advantages in terms of test readability, assertion generation quality, and complex logic coverage. This section reviews the related work from three perspectives: methods based on static analysis [3], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], methods that rely entirely on LLMs [35], [36], [37], [38], [39], [40], [41], and hybrid approaches that integrate static analysis with prompt learning [42], [43], [44], [45].

A. UNIT TEST CASE GENERATION TECHNIQUES BASED ON STATIC ANALYSIS

Traditional unit test case generation methods primarily rely on static analysis techniques. Without requiring actual program execution, these methods model the structural and semantic information of source code to extract key elements such as control flow, data flow, dependency relationships, and logical constraints, thereby constructing test inputs that satisfy specific coverage criteria. The main categories of static analysis methods include the following:

1) STRUCTURAL ANALYSIS

This approach extracts basic program elements such as classes, methods, parameters, and modifiers through Abstract Syntax Tree (AST) parsing. Oumarou and El Mansour [3] performed static analysis on the methods under test by constructing ASTs to extract key information such as variables, constants, method parameters, and control structures. Based on the identified variables and their inferred types within the AST, random test inputs were generated and composed into object instances, thereby enabling the automatic generation of unit test methods. This process does not require program execution and achieves semi-automated test data generation solely by relying on syntactic structures. Bozga et al. [4] conducted structural static analysis on protocol specifications prior to test case generation and employed slicing techniques to reduce model size. Specifically, they first applied control flow analysis to prune unreachable states and transitions, followed by data flow analysis to eliminate variables and assignments unrelated to test outputs, and finally used constraint propagation to remove branches that could never be triggered. The resulting reduced model retains semantic equivalence with the original testing objectives while significantly mitigating state space explosion, thus enabling more efficient generation of unit test cases. Joshy et al. [5], through structural analysis in static analysis, proposed a syntax correction algorithm based on the Lowest Common Ancestor (LCA) approach to repair non-compilable code fragments reported by static analysis tools. The repaired code fragments become syntactically valid and semantically preserved minimal executable segments. On this basis, the authors further generated test stubs and inputs for the repaired segments and executed them using dynamic testing tools (such as fuzzing,

KLEE, and Valgrind), thereby transforming static warnings into verifiable unit test cases and enabling automated validation of true and false positives.

2) CONTROL FLOW ANALYSIS AND DATA FLOW ANALYSIS

This approach guides the generation of test paths by constructing Control Flow Graphs (CFGs) and identifying Def-Use Chains. Zhang et al. [6] applied the All-Uses data flow criterion by building CFGs for Solidity smart contracts and performing data flow analysis to extract all definition-use pairs (DUPs), including critical paths related to require statements. Subsequently, a Genetic Algorithm (GA) was used to optimize test case generation, with an improved fitness function guiding the testing process to enhance DUP coverage, particularly strengthening the testing of require condition paths. Su et al. [7] identified def-use pairs in programs through static analysis and treated them as targets for data flow testing. They employed iterative data flow analysis and dominator analysis to construct control flow graphs and determine critical path points (cut-points). These static features were used to guide symbolic executors in prioritizing the exploration of paths more likely to cover the test targets, thereby efficiently generating unit test cases that satisfy data flow coverage criteria. Su et al. [8] conducted a systematic review of the field of Data-Flow Testing (DFT), categorizing and summarizing five mainstream test case generation methods—search-based, random, accompanying-coverage, symbolic execution, and model checking—based on the three core phases of def-use pair identification, test data generation, and coverage tracking. The study highlighted the advantages of data-flow testing in improving defect detection capabilities while also pointing out its limitations in scalability and unreachable target identification, calling for future research to further enhance its practicality and automation. Babić et al. [9] combined dynamic execution traces with static completion to construct interprocedural control flow graphs, and represented the program's global control flow structure using Visibly Pushdown Automata (VPA), falling under control flow/structural analysis. Static analysis was then performed on the VPA to locate potential vulnerabilities and compute data-dependency slices that backward-propagate from vulnerable statements, which served as goals and constraints for path exploration—belonging to data flow analysis. Finally, symbolic execution was guided toward these target points using control flow distance to the goal and slice coverage as heuristics, thereby generating test cases capable of triggering the desired behaviors.

3) DEPENDENCY ANALYSIS

This approach models inter-class relationships using call graphs and inheritance hierarchies to support the generation of test targets for complex object-oriented programs. Derakhshanfar et al. [10] proposed CLING, an integration testing generation tool based on static analysis and search-based test generation. CLING performs static analysis to construct a class-level Concurrent Control Flow

Graph (CCFG), identifies method invocation relationships between caller and callee classes, and extracts interaction points (call sites) from the call graph to guide test target generation. Additionally, to handle inheritance and polymorphism, CLING distinguishes between method overrides in parent and child classes and adjusts test target classes and invocation paths accordingly to ensure the coverage of dynamically bound behavior in inheritance structures. By combining call graph analysis with inheritance hierarchy modeling, this method effectively generates integration test cases that cover real inter-class dependencies.

4) SYMBOLIC EXECUTION ANALYSIS

This approach models path conditions and uses constraint solvers to derive feasible inputs. Godefroid [11] employed symbolic execution to symbolically model program paths and used constraint solvers to generate concrete inputs that satisfy path constraints, thereby driving the program to cover different execution paths and enabling the generation of unit test cases. In this process, static symbolic execution directly analyzes code to produce test inputs, while dynamic symbolic execution incorporates runtime information to mitigate the limitations of static analysis, thus improving both coverage and practicality. Uehara [12] abstracted program inputs into symbolic variables through symbolic execution, recorded path conditions during branch traversal, and used SMT solvers to generate concrete input values satisfying those conditions, thereby obtaining executable test cases. This method enables systematic coverage of feasible paths in the program and supports automated generation of unit test cases. Staats and Păsăreanu [13] utilized Symbolic PathFinder (SPF) to perform symbolic execution on Java bytecode, representing input variables as symbolic values and generating path conditions at branch points. These conditions were then solved using constraint solvers to obtain concrete inputs for different execution paths, producing unit test cases that cover the program structure. In addition, the authors proposed a Simple Static Partitioning (SSP) method to statically partition the symbolic execution tree and explore branches in parallel, thereby improving the efficiency of symbolic execution in automated test case generation. Pham et al. [14] introduced a symbolic execution method based on separation logic, modeling path conditions in heap-structured programs as logical formulas that include pointer relations, recursive predicates, and numerical constraints. The S2SATSL solver was then used to analyze the satisfiability of these path conditions. Through context-sensitive lazy initialization and least fixed-point analysis, the method explores only those feasible paths that satisfy the expected structural constraints, thereby generating structurally valid and thoroughly covering unit test cases.

These static analysis-based test case generation methods achieve high-coverage and high-determinism test objective generation by modeling the structural and logical behaviors of programs without requiring actual program execution. Their advantages lie in strong controllability, logical

traceability, and stable outcomes, making them particularly suitable for testing programs with well-defined structures and deterministic behaviors. However, such methods typically rely on heuristic strategies for input construction and assertion generation, lacking semantic contextual understanding and reasoning capabilities. As a result, they struggle to automatically generate test cases with high readability and semantic completeness, which limits their practicality and level of automation. Consequently, with the rapid advancement of LLMs, researchers have begun exploring their integration into test case generation tasks. By leveraging the powerful natural language and code understanding capabilities of LLMs, this line of research aims to address the limitations of traditional static analysis and to explore more efficient and intelligent solutions for automated test generation.

B. UNIT TEST CASE GENERATION TECHNIQUES ENTIRELY DEPENDENT ON LLMs

With the increasingly powerful performance of LLMs in natural language processing and source code understanding tasks, some researchers have completely abandoned the static analysis typically required by traditional test case generation techniques and instead attempt to rely solely on the inherent capabilities of LLMs in processing natural language and source code to generate unit test cases. These approaches typically guide the model to produce complete test code—including setup and assertion statements—by designing high-quality prompts, thereby emphasizing the model's generalization ability and contextual understanding capacity.

In the context of method-level test generation, Yuan et al. [35] proposed a method that leverages a basic prompt design, in which the focal method and its related code context are directly provided to ChatGPT [18], enabling the model to generate complete unit test cases that include both the test prefix and test oracle. To enhance the quality of test generation, the authors further introduced the ChatTester framework, which adopts an “intent inference + iterative correction” workflow. Specifically, intent prompts are first used to help ChatGPT understand the target method, followed by iterative refinement of the generated test code based on compilation error messages, thereby improving the correctness and usability of the test cases. Li and Yuan [36] leveraged LLMs as test agents by directly guiding the model through prompt instructions to generate function-level input-output paired test cases in the form of assert statements, and subsequently evaluated their quality. To enhance the accuracy of test generation, they proposed the TestChain framework, which decomposes the test case generation process into two sub-tasks: test input generation and test output generation. This decomposition significantly improves the LLMs' capability to generate high-quality test cases for complex scenarios. Liu et al. [37] proposed the AID approach, which integrates LLMs with differential testing to generate unit test cases. Specifically, the method first utilizes LLMs, guided by program specifications and the target program, to generate multiple program variants. Then, LLMs are employed to

construct input generators capable of producing valid test inputs. Finally, differential testing is used to analyze discrepancies in the outputs of the program variants and the original program, identifying defect-triggering inputs and the corresponding test oracles. This process ultimately yields test cases capable of revealing defects, effectively improving the precision and practical utility of the generated test cases.

In the domain of class-level test automation, Lops et al. [38] employed LLMs to generate class-level test cases for Java through automated prompt engineering. Their approach supports both zero-shot and few-shot prompting strategies. The prompts include contextual information such as the target class code, testing framework, and Java version, thereby guiding the LLM to automatically produce the corresponding test class code. The entire process is integrated into the AgoneTest system, which enables large-scale, automated invocation of LLMs, parsing of generated outputs, and integration of test cases into the target project—ultimately achieving fully automated generation and evaluation of unit test cases. Sapozhnikov et al. [39] integrated LLMs into the IntelliJ IDEA plugin TestSpark to automatically generate unit test cases for Java programs. The plugin constructs prompts containing information such as source code, input signatures, and polymorphic relationships based on the target unit (class, method, or code line), and sends them to the LLMs. The generated test code is then received and parsed by the system. Additionally, a feedback loop mechanism is introduced to automatically correct compilation errors and iteratively refine the generated results, ensuring that the test cases are compilable and executable.

In the area of industrial-grade tool development, Zhou [40] presented the SmartUnit framework at the 2023 AiDD Summit. Developed by Ant Group, this tool leverages AIGC (Artificial Intelligence Generated Content) technologies to automatically generate, execute, filter, and manage unit test cases, thereby improving testing efficiency and coverage quality. The overall architecture of SmartUnit is composed of several modules, including model training, test case generation, validation and execution, and test case pruning, enabling full lifecycle management of unit testing. Its key innovation lies in the integration of LLMs-based generation with rule-guided strategies, achieving the automatic construction of high-quality, compilable, and assertion-rich test cases. The framework has been validated and applied in real-world enterprise-level projects. Schäfer et al. [41] developed the TestPilot tool, which generates JavaScript unit test cases by invoking pre-trained LLMs (e.g., GPT-3.5-turbo) using prompts that include the function signature, documentation comments, usage examples, and function source code. The generated test cases are executed automatically, and in the event of test failures, the system feeds the failing test cases and corresponding error messages back into the model as new prompts. This enables the model to adaptively revise and improve the test cases, thus iteratively producing high-quality tests.

The aforementioned techniques leverage the powerful natural language and source code understanding capabilities of LLMs to greatly simplify the test case generation process, thereby enhancing the level of automation and improving development efficiency. However, such approaches commonly rely on the model's direct response to prompts and lack deep modeling and utilization of structured information in source code. As a result, the generated test cases often suffer from deficiencies in coverage completeness, logical rigor, and adaptability to complex code scenarios. This limitation underscores the necessity of integrating traditional static analysis techniques with prompt learning using LLMs to further improve the quality and practical value of test case generation.

C. UNIT TEST CASE GENERATION TECHNIQUES INCORPORATING TRADITIONAL STATIC ANALYSIS

Some researchers have attempted to incorporate traditional static analysis as an auxiliary step in the process of generating test cases based on LLMs.

The first category of methods focuses on enhancing the model's understanding of code structure and semantics through static analysis. Chen et al. [42] proposed the ChatUniTest framework, which adopts an adaptive focal context mechanism to dynamically extract key contextual information from the target code and construct high-quality prompts for input into LLMs, thereby guiding the generation of highly readable and interpretable unit test cases. After generation, the framework employs a generate–verify–repair workflow, sequentially performing syntax, compilation, and runtime verification, and leverages both rule-based strategies and LLMs for automated repair, enhancing the correctness and practicality of the generated test cases. Wu et al. [43] improved unit test case generation by integrating traditional path-sensitive analysis techniques with large language models (LLMs), such as CodeX [16] and StarCoder [28]. They proposed a Path-Aware approach that leverages static analysis to construct program execution path information as contextual prompts, thereby enhancing the LLMs' comprehension of program structure and behavior. During the generation process, the statically extracted path information is encoded into natural language prompts and incorporated into the LLMs, guiding the model to produce test cases with stronger semantic validity and structural coverage. This approach exemplifies the synergistic integration of static analysis and LLM capabilities. Roy et al. [44] employed static program analysis to extract key information about the target method—such as method signatures, invoked methods, and field types—and used this concise and accurate context as input prompts to the LLM, effectively mitigating the issue of context overflow caused by redundant code. By combining the JUnit [55] and Mockito frameworks, the LLM, under the guidance of these enhanced prompts, generates syntactically correct, compilable unit test cases with assertions, significantly improving the success rate of test generation.

The second category of methods integrates slicing techniques with CoT prompting, enabling more precise and flexible control over the granularity of static analysis and facilitating finer-grained test case generation. Zhang et al. [45] adopted a method slicing approach, decomposing complex methods into multiple logical slices, and then guided LLMs with CoT prompts to generate test cases for each slice individually, ensuring full line and branch coverage. These slice-level test cases are subsequently combined into a complete test suite. During the generation process, static analysis is incorporated to extract contextual information to assist LLMs comprehension, and Self-Debug techniques are employed to automatically repair test cases that fail to execute, thereby improving overall coverage and executability.

These techniques leverage static analysis to extract key contextual information, control flow, and data flow characteristics from the source code, incorporating such information into the prompt design as auxiliary input. This integration enhances the large language model's ability to comprehend code structure and semantics, thereby improving the quality and coverage of generated test cases. However, in existing approaches, static analysis and prompt design are typically loosely coupled, lacking deeply coordinated optimization tailored to specific testing objectives. Moreover, most prompting strategies do not systematically explore the synergy between static analysis insights and CoT prompting techniques. In addition, the extraction of static analysis information remains incomplete and unsystematic. As a result, the generated test cases still exhibit limitations in terms of logical rigor, coverage completeness, and automated repair capabilities, which warrant further improvement.

D. GAP ANALYSIS

In summary, existing approaches to unit test case generation have achieved notable progress in terms of automation and practical applicability. However, they still exhibit significant limitations. Traditional methods based on static analysis—such as constructing control flow graphs, data flow graphs, or symbolic paths—can achieve high structural coverage and logical precision. Nevertheless, these approaches often rely on manually crafted rules or heuristic strategies for input construction and assertion generation. They lack semantic-level reasoning capabilities, making it difficult to produce complete test cases that are both readable and executable. Prompt-only strategies, while exhibit a degree of generality and code comprehension capability—enabling adaptation to diverse testing scenarios without complex configurations often lack a deep understanding of code structure, which may result in incomplete or incorrect test cases. In contrast, existing approaches that incorporate static analysis with prompt learning demonstrate improved generation quality to some extent, but their extraction of analysis information is often incomplete. These methods have not yet fully accounted for critical code information, inter-class dependencies, package structure, or the varying static characteristics associated with different coverage objectives. Furthermore, prompt design is

generally not tightly integrated with static analysis. Therefore existing approaches still exhibit critical research gaps in several key dimensions, including structural modeling capability, prompt structure design, adaptability to multi-dimensional coverage objectives, and guarantees of test executability. To address these limitations, this paper proposes the IntelliUnitGen framework, which aims to deeply integrate static analysis with prompt learning. The framework constructs structured, coverage-oriented prompt inputs and introduces a CoT prompting mechanism to comprehensively enhance the accuracy, coverage, compilability & executability of the generated test cases. The IntelliUnitGen framework automatically extracts multi-granularity structural information—such as inter-class dependencies, method invocation relationships, control flow, and data flow—through static analysis. It then performs feature selection and reorganization based on the desired coverage objectives, injecting both structured and semantic key information into the prompt construction process. During the prompt learning phase, IntelliUnitGen employs multi-stage structured prompt templates and a CoT reasoning strategy to guide large LLMs in stepwise generation of input construction, assertion design, and test logic, thereby achieving a deep synergy between structural comprehension and semantic generation. In addition, the framework incorporates an automated post-generation validation, regeneration, and manual repair pipeline to ensure the compilability & executability of the generated test cases while supporting various test coverage criteria.

III. METHODOLOGY AND OVERALL WORKFLOW OF THE INTELLIUNITGEN FRAMEWORK

A. INTELLIUNITGEN METHODOLOGY

The design of the IntelliUnitGen framework integrates the strengths of static analysis and LLM-based prompt learning. Its underlying methodology is primarily grounded in three key pillars: theoretical foundations, practical experience, and empirical observations.

First, from a theoretical perspective, numerous recent studies have demonstrated that LLMs possess strong capabilities in semantic understanding and natural language generation. However, in the absence of structured information, LLMs tend to generate test cases that are incomplete, non-compilable, or lack assertions [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], [43], [44], [45], [46]. In contrast, traditional static analysis techniques, grounded in rigorous control flow and data flow modeling, enable the systematic extraction of program structure and logical path information, providing solid support for test input generation [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]. Building upon the complementary strengths of these two paradigms, IntelliUnitGen proposes a hybrid approach that statically extracts class structures, dependency relationships, and conditional paths, and integrates CoT prompting [46] to guide step-by-step reasoning in LLMs, thereby enhancing the accuracy and executability of the generated test cases.

Second, in engineering practice, researchers have observed during the initial attempts to generate test cases using LLMs that even when standard prompts—including method signatures and comments—are provided, the models may still overlook boundary conditions, logical paths, or produce invalid assertions. To address this issue, we incorporate a static analysis module into the prompt construction process, which assists in identifying critical execution paths, dependent fields, and exception-handling branches within the program. These elements are then structurally transformed into context-enhanced prompt inputs to improve the model's structural awareness of the code under test.

Specifically, during the static analysis phase, IntelliUnitGen leverages the JavaParser tool to extract inter-class relationships, including method invocations, inheritance hierarchies, and field dependencies. These dependencies are transformed into structured contextual information and embedded into the prompt input. For instance, when a method in class *A* depends on objects or methods from class *B*, the framework explicitly presents the initialization process and accessible interfaces of class *B* within the prompt. This design guides large language models (LLMs) to generate test code that is aware of cross-class dependencies. As a result, the framework can produce more reasonable, compilable, and executable test cases, particularly in scenarios involving complex inter-class interactions.

Finally, through experimental observations on several real-world projects (e.g., Commons CLI [47], Commons CSV [48], Binance-Connector [49]), we found that static analysis information significantly enhances prompt design. For instance, when dependency classes and field descriptions are not provided, LLMs often generate test code with missing logic or failed object instantiations. However, upon introducing class dependency structures, both the generation accuracy and compilation success rate are substantially improved. Consequently, we further incorporate a modular static analysis process to uniformly abstract information such as class structures, method dependencies, and conditional branches, and construct corresponding structured prompt inputs based on specific testing goals (e.g., statement coverage, condition coverage).

In summary, the design of IntelliUnitGen is not merely empirically driven, but is grounded in a triad of theoretical foundation, practical feedback, and empirical validation. It aims to achieve high-quality, coverage-oriented automated test case generation through fine-grained static information modeling and step-by-step reasoning prompt strategies.

B. OVERALL WORKFLOW OF THE INTELLIUNITGEN FRAMEWORK

The overall workflow of the IntelliUnitGen framework is illustrated in Fig. 1. IntelliUnitGen integrates static analysis with LLMs, incorporates CoT reasoning, and employs step-by-step prompting to generate test cases, thereby enhancing the model's comprehensive capabilities in understanding complex logic, generating accurate test cases, and achiev-

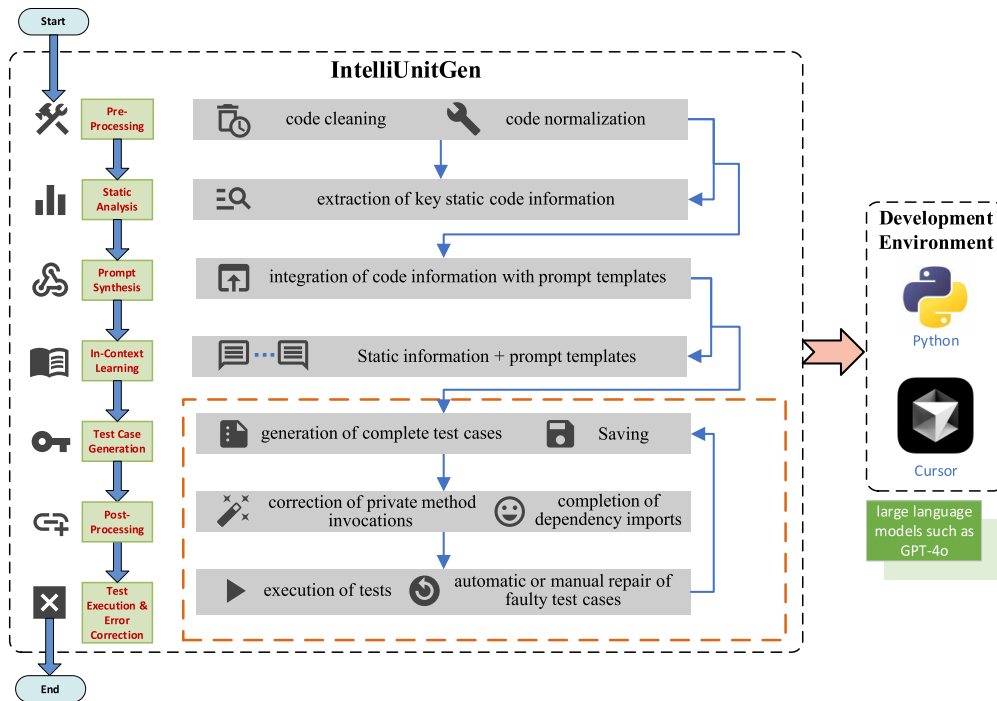


FIGURE 1. Overall framework of IntelliUnitGen.

ing higher coverage. The framework is divided into seven stages: Pre-Processing, Static Analysis, Prompt Synthesis, In-Context Learning, Test Case Generation, Post-Processing, and Test Execution & Error Correction. These stages are respectively responsible for: (1) cleaning and normalizing the source code; (2) extracting key static information from the code; (3) synthesizing this information with prompt templates into concrete prompts; (4) guiding LLMs to learn the style and structure of test cases; (5) generating complete test cases and saving them; (6) correcting private method calls and completing dependency imports; and (7) executing the tests and automatically or manually fixing any erroneous test cases.

IV. KEY TECHNIQUES AND IMPLEMENTATION DETAILS OF THE INTELLIUNITGEN FRAMEWORK

A. PRE-PROCESSING STAGE

The primary goal of the pre-processing stage is to reduce redundancy and complexity in the source code, optimize the code structure to make it more concise and clear, and thereby enhance the ability of LLMs to understand the source code.

IntelliUnitGen selects the target Java code through a file selection dialog and performs code pre-processing, which includes the following steps:

1) CODE CLEANING

Remove comments and blank lines, eliminate extra spaces (mainly including spaces around operators, parentheses, and semicolons), to make the source code more concise and reduce the interference of redundant information on LLM.

2) CODE NORMALIZATION

Unify the indentation style to K&R style (Kernighan & Ritchie style), adjust statements split across multiple lines into a single line, or separate multiple statements written in a single line into multiple lines.

The above series of preprocessing operations can effectively expands the upper limit of code scale that LLMs can process. During the experimental process, we observed that preprocessing has a critical impact on the ability of LLMs to parse complex and large-scale source code. For example, in the dataset commons-cli-master [47], the original code of the CommandLine class contains as many as 1007 lines. If the unprocessed complete code is directly embedded into the prompt and passed to the LLM, the input content will exceed the model's processing window, resulting in the inability to return any valid output. However, after introducing the pre-processing mechanism—by removing redundant comments, eliminating blank lines, and unifying formatting—the number of code lines is significantly compressed to fall within the acceptable range of the LLM. As a result, the model successfully generated structurally sound test cases. More importantly, the normalized code input exhibits improved readability, consistency, and contextual adaptability, which reduces the risk of semantic misinterpretation by the model when understanding complex logic, thereby enhancing the applicability, executability, and coverage quality of the generated test cases.

B. STATIC ANALYSIS PHASE

In the static analysis phase, the system parses the entire test project and the class under test to extract key information,

including the class name, number of methods, input parameters and modifiers of each method, decision-making nodes, exception handling constructs, and other fundamental elements. Additionally, it captures the dependency relationships between the class under test and other classes within the project, as well as the package information of the test project. This process not only facilitates the optimization of code representation and enhances the LLM's ability to understand code logic, but also lays a solid foundation for the subsequent generation of test cases and coverage analysis.

In addition, during this phase, it is also necessary to extract corresponding code information for different coverage standards, including statement coverage, decision coverage, condition coverage, and branch coverage:

1) STATEMENT COVERAGE

Counts the number of executable statements in the Java code to ensure that test cases are capable of executing all executable statements.

2) DECISION COVERAGE

Extracts all control flow decision points (such as if-else statements and loop control structures) to ensure that test cases cover all possible execution paths.

3) CONDITION COVERAGE

Analyzes Boolean expressions and extracts the number of independent conditions to ensure that both the True and False outcomes of each condition are thoroughly tested.

4) BRANCH COVERAGE

Identifies all branch paths within control structures (such as if, else, switch-case, and ternary operators), and extracts the trigger conditions corresponding to each branch to ensure that test cases can cover every possible branch execution path.

Through these analyses, IntelliUnitGen is capable of accurately identifying critical execution paths within the code, thereby assisting LLMs in generating more comprehensive and rigorous test cases, ultimately enhancing test coverage and the effectiveness of code verification.

C. PROMPT SYNTHESIS PHASE

In the prompt synthesis phase, IntelliUnitGen integrates the key information required for test case generation with pre-defined prompt templates and submits them to LLMs for processing, in order to generate test cases that meet the specified testing criteria.

IntelliUnitGen employs a step-by-step prompting strategy to guide LLMs in analyzing, reasoning, and generating test cases. Compared with one-shot generation approaches, the step-by-step prompting method reduces the risk of LLMs missing edge cases and critical logical branches during the generation process, thereby improving the completeness and coverage of the generated test cases. The design rationale for the five types of coverage prompts is as follows:

1) STRUCTURED INPUT IN COORDINATION WITH STATIC ANALYSIS

All prompt templates are based on the results of static analysis and present the key elements of the code—such as class names, method names, parameter lists, decision nodes, exception structures, dependency relations, and method signatures—in a structured format. This enhances the model's global perception and semantic understanding of the code under test. The structured input ensures contextual completeness of the prompts and helps guide the model to generate more targeted and relevant test logic.

2) TASK DECOMPOSITION AND STEP-BY-STEP REASONING GUIDANCE

Instead of issuing direct generation requests to the model, IntelliUnitGen adopts a step-by-step prompting strategy that decomposes the test generation process into multiple subtasks—such as structural understanding, statement extraction, input construction, and test generation. This multi-turn guidance enhances the reasoning quality of the model. Since different types of coverage tasks have distinct requirements, the design of each step is significantly differentiated accordingly.

3) CUSTOMIZED DESIGN FOR COVERAGE OBJECTIVES

① Statement Coverage: The model is guided to identify all executable statements and construct input combinations that can trigger each statement, thereby ensuring that every line of code is covered by the generated test cases.

② Condition Coverage: The model is prompted to decompose boolean expressions into independent boolean factors, analyze the behavioral differences under both True and False conditions, and generate corresponding test cases to cover all logical combinations.

③ Branch Coverage: The model is guided to identify branch paths within control structures (e.g., if-else, switch-case), analyze the triggering conditions of each path, and generate input sets capable of covering all possible branches.

④ Decision Coverage: With a focus on the boolean outcomes of entire decision statements, the model is prompted to construct inputs that cause the decision to evaluate to both True and False, thereby ensuring dual-branch coverage of the overall decision.

⑤ Path Coverage: Combined with the extraction of cyclo-matic complexity from decision nodes [54]:

$$\begin{aligned} & \text{Loop Complexity} \\ &= \text{Number of Decision Nodes} + 1 \end{aligned} \quad (1)$$

Guide the model to reason through all possible independent paths and design triggering conditions to achieve comprehensive coverage of normal flows, exceptional paths, and boundary paths.

D. IN-CONTEXT LEARNING STAGE

The primary objective of the in-context learning module is to enhance the LLMs' understanding of the style and logic of test cases, ensuring that the generated test cases conform to the specifications of the designated testing framework. Before prompting the LLMs to reason and generate test cases, we provide example code along with corresponding standard test cases, enabling the LLMs to learn the correct format, key identifiers, and structure of test cases. This allows the models to automatically adhere to these standards during the generation process, thus avoiding disorganized formats or non-compliance with industry norms in the generated test code.

For example, different testing frameworks have distinct key identifiers: the JUnit framework [55] and TestNG framework [56] use `@Test` as the annotation for test methods; the Mocha framework [57] structures tests using the `describe` and `it` keywords; the unittest framework [58] requires test classes to inherit from `unittest.TestCase` and test methods to be prefixed with `"test_"`; whereas the Pytest framework [59] and the Testing framework [60] require test functions to begin with `"test_"` and `"Test"`, respectively. If we do not provide exemplars to the LLMs, the model may incorrectly adopt test styles from other programming languages (e.g., applying Python's unittest or Pytest styles to Java code), thereby generating test cases that are incompatible with the technical stack of the target code, which increases the difficulty of subsequent validation and execution.

By introducing in-context learning, we are able to guide LLMs to automatically follow the format specifications of the designated testing framework during test case generation. This enhances the consistency and executability of the generated test code, thereby optimizing the performance of IntelliUnitGen in the task of automated test case generation.

E. TEST CASE GENERATION PHASE

The test case generation phase constitutes the core component of the IntelliUnitGen framework. Following prior stages of static analysis, prompt synthesis, and in-context learning, the system invokes the large LLM API to process structured prompt inputs. The LLMs are guided step-by-step to analyze program logic, infer valid test inputs, and iteratively generate preliminary test cases in accordance with different coverage criteria. Subsequently, the model considers boundary conditions and special cases to automatically generate test case code that satisfies both semantic constraints and coverage objectives.

After the LLMs return the generated test cases, the system automatically extracts the pure code segments and saves them to the directory specified by the user through an interactive window at system startup.

F. POST-PROCESSING PHASE

After the generation of test cases, it is necessary to perform essential post-processing on the generated test code to

enhance its executability and engineering quality. This phase primarily includes the following two operations:

1) REFLECTION HANDLING FOR PRIVATE METHODS

Apply reflection techniques to private method calls within the test class to ensure that private methods in the class under test can be correctly accessed and invoked during testing.

2) AUTOMATIC ADDITION OF PACKAGE AND IMPORT STATEMENTS

Automatically analyze the project structure and add the necessary package declarations and import statements to the test class to ensure that all required dependency classes are correctly referenced.

These two steps jointly ensure that the generated test cases can be correctly compiled and executed.

G. TEST EXECUTION AND ERROR CORRECTION PHASE

In the test execution and error correction phase, IntelliUnitGen automatically detects syntactic and semantic errors in the generated test cases to ensure that they can be successfully compiled and executed. This phase does not focus on test coverage metrics; instead, it emphasizes the correctness and executability of the code, ensuring that the test cases can run smoothly in practical testing environments. The steps in this phase are as follows:

1) EXECUTION OF TEST CASES

After saving the generated test cases, IntelliUnitGen executes the compilation command via the specified Maven path (`maven_path`) to detect any syntactic or semantic errors in the code. If the compilation succeeds, the system will automatically execute and evaluate the test cases, generating a test coverage report.

2) TEST CASE ERROR CORRECTION

If the initially generated test cases still fail to compile or execute after post-processing, the system will attempt up to fifteen rounds of test case regeneration. If all fifteen attempts fail to produce a valid test case, the process will be terminated, and a failure log will be generated to facilitate subsequent manual error correction.

V. ILLUSTRATIVE EXAMPLE

To provide a more intuitive demonstration of how IntelliUnitGen operates during the test case generation process and to highlight its advantages, this section presents a representative example illustrating the complete workflow across key stages, including code preprocessing, static analysis, prompt construction, test case generation, compilation, and regeneration. In both the illustrative example and subsequent experiments, the large language model employed is GPT-4o. As shown in Fig. 2, the Java code under test used in the illustrative example is designed to calculate a discount based on the purchase amount, where different consumption ranges correspond to different discount rates.

```

1 package test;
2
3 public class DiscountCalculator {
4     public double calculateDiscount(double amount) {
5         if(amount < 0) {
6             throw new IllegalArgumentException("Amount cannot be negative");
7         } else if(amount >= 1000) {
8             return amount * 0.2;
9         } else if(amount >= 500) {
10            return amount * 0.1;
11        } else {
12            return amount * 0.05;
13        }
14    }
15 }
16
17 }

```

FIGURE 2. Sample Java code under test.

```

##### Phase 1: Preprocessing #####

Recognized {len(analysis.function_list)} methods: 1

package java_code;
public class DiscountCalculatorMaven {
    public double calculateDiscount(double amount){
        if(amount<0){
            throw new IllegalArgumentException("Amount cannot be negative");
        } else if(amount>=1000){
            return amount*0.2;
        } else if(amount>=500){
            return amount*0.1;
        } else {
            return amount*0.05;
        }
    }
}

```

FIGURE 3. Outputs of the pre-processing stage.

A. PREPROCESSING STAGE OF THE ILLUSTRATIVE EXAMPLE

In the preprocessing stage, IntelliUnitGen performs code cleaning and normalization to make the code more concise, standardized, and structurally clear, thereby facilitating better understanding by the large language model. The output of the preprocessed code under test is shown in Fig. 3.

B. STATIC ANALYSIS STAGE OF THE ILLUSTRATIVE EXAMPLE

In the static analysis stage, code-level information applicable to various coverage types, as well as specific information corresponding to different coverage criteria, is extracted. The result of the static analysis for statement coverage based on the aforementioned code under test is shown in Fig. 4.

C. PROMPT COMPOSITION PHASE OF ILLUSTRATIVE EXAMPLE

The prompt composition phase integrates the results of the above analyses with predefined prompt templates. The output

of IntelliUnitGen at this stage is the composed prompt used to generate test cases that satisfy the requirement of statement coverage for the target code.

=====
Step 0.1: System Instruction
=====

[System Prompt] You are an assistant known for detailed explanations and clear logic. Your responses should be warm in tone and contain thorough reasoning steps and complete answers.

=====
Step 0.2: System Instruction
=====

[System Prompt] You are a system analyst specializing in software testing and static analysis. Please assist me by responding to the following series of prompts.

=====
Step 1: Understanding Code Structure
=====

[Structural Information Prompt]

The following is the result of static analysis on the Java code structure:

Class name: DiscountCalculatorMaven

```
#####Phase 2: Static Analysis#####

@@@@ Dependency Information @@@@@

all_dependency_result: {'DiscountCalculatorMaven': [], 'Review_User': []}
class_file_name: DiscountCalculatorMaven
[]

@@@@@ Extracted Method Signature @@@@@

{}

@@@@@Specific Information of Code@@@@@

Number of Executable Statements: 4

Extracted Executable Statements: [''] else if(amount>=1000){\n\t\t\t\treturn amount*0.2;','} else if(amount>=500){\n\t\t\t\treturn amount*0.1;','} else {\n\t\t\t\treturn amount*0.05;','}\n\t\t\t\t\n;''

@@@@@Common Information of Code@@@@@

Extracted Decision Nodes: : ['amount<0', 'amount>=1000', 'amount>=500', 'amount>=1000', 'amount>=500']

Extracted Exception Handling: : []

Extracted Class Name: : ['DiscountCalculatorMaven']

Extracted Method Name: : [['calculateDiscount', 'double amount']]

Extracted Method Info:: [{ 'function_name': 'calculateDiscount', 'parameters': ['double amount'], 'modifier': 'public' }]
```

FIGURE 4. Outputs of the static analysis stage.

Number of methods: 1, method name: ['calculateDiscount']

Input parameters and modifiers for each method:

[{'function_name': 'calculateDiscount', 'parameters': ['double amount'], 'modifier': 'public'}]

Decision nodes:

['amount<0', 'amount>=1000', 'amount>=500', 'amount>=1000', 'amount>=500']

Exception handling structures: []

Java source code: #####

package java_code;

```
public class DiscountCalculatorMaven {
    public double calculateDiscount(double amount){
        if(amount<0){
            throw new IllegalArgumentException
                ("Amount cannot be negative");
        } else if(amount>=1000){
            return amount*0.2;
        } else if(amount>=500){
            return amount*0.1;
        } else {
            return amount*0.05;
        }
    }
}
```

#####

===== Step 2: Understanding Dependencies
=====

[Dependency Analysis Prompt]

The following is the result of dependency analysis for the Java code:

Dependent classes: []

Method signatures of dependent classes: {}

Constructors of dependent classes: {}

===== Step 3: Analyzing Executable Statements
=====

[Executable Statement Analysis Prompt] According to the above code structure, there are 4 executable statements. Please enumerate all executable statements within the method (excluding declarations, comments, and blank lines) and assign them statement numbers.

===== Step 4: Analyzing Execution Conditions
=====

[Execution Condition Analysis Prompt] Based on the executable statements listed above, please design at least one input parameter combination for each statement to ensure that it will be executed.

===== Step 5: Generating Test Cases
=====

[Test Case Generation Prompt] Based on the analysis above, refer to the following example and its corresponding test case. Generate a JUnit 5 test class named DiscountCalculatorMavenTest.java, ensuring 100% statement coverage:

#####

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

import static org.junit.jupiter.api.Assertions.*;


```
import org.junit.jupiter.api.Test;

public class CalculatorTest {
    @Test
    public void testAdd() {
        Calculator calculator = new Calculator();
        assertEquals(5, calculator.add(2, 3));
    }
}

#####
Target code to test:
#####
package java_code;
public class DiscountCalculatorMaven {
    public double calculateDiscount(double amount){
        if(amount<0){
            throw new IllegalArgumentException("Amount cannot be
negative");
        } else if(amount>=1000){
            return amount*0.2;
        } else if(amount>=500){
            return amount*0.1;
        } else {
            return amount*0.05;
        }
    }
}

#####
Test Case Generation Requirements and Constraints:
```

- (1) When generating unit test cases, avoid passing null as arguments to overloaded methods. Prefer explicitly typed parameters or alternative values to prevent compilation errors.
- (2) If the constructor of the target class is private or protected, do not instantiate it using new ClassName(...). Instead, use the builder pattern provided by the class. For example, use ClassName.Builder builder = new ClassName.Builder(); followed by calls to addXxx(...) or setXxx(...), and finally call builder.build(). Ensure correct builder methods are used based on the parameters, and that the generated code is legal and compilable.
- (3) Under no circumstances should non-public fields, methods, or constructors be accessed in the generated test code. This includes default static fields like DEFAULT, EMPTY, and INSTANCE, which must not be referenced unless explicitly declared public. All test objects must be constructed using publicly accessible APIs or builder patterns.
- (4) Generic type casting using syntax like (T) is strictly prohibited. All type conversions must use clearly declared, concrete types.
- (5) Ensure the generated test cases conform to the actual behavior of the target method. If the method returns null to indicate "unsupported," use assertNull rather than forcibly applying assertsThrows.

- (6) All classes used in the test cases must have appropriate import statements added, including standard library classes (e.g., List, ArrayList, Map, Properties) and project-specific classes (e.g., CommandLine, ParseException). Omitting these imports can lead to "unresolved symbol" or "not imported" compilation errors.
- (7) When using SimpleDateFormat.parse(...), do not pass in strings that do not match the expected format. If testing for format mismatch, explicitly assert exceptions using assertsThrows(ParseException.class,...) to prevent test failure.
- (8) Do not use Mockito or any other mocking tools. Avoid mocking final classes, system classes, or third-party libraries. All test objects must be instantiated with real data or actual instances. Mocking behavior is strictly prohibited.

D. CONTEXT LEARNING, TEST CASE GENERATION, AND POST-PROCESSING PHASE OF THE ILLUSTRATIVE EXAMPLE

In the test case generation phase, the prompt generated in the previous stage is passed to the LLMs to generate test cases, followed by post-processing. The output for statement coverage corresponding to the target code is shown in Fig. 5.

E. TEST EXECUTION AND ERROR CORRECTION PHASE OF THE ILLUSTRATIVE EXAMPLE

In this case, the generated test cases executed successfully on the first attempt, thus no error correction was required. The test coverage measured by the JaCoCo tool reached 100%. The coverage report is shown in Fig. 6.

VI. EXPERIMENT AND RESULT ANALYSIS

To evaluate the effectiveness of IntelliUnitGen in automated test case generation, this study conducts experiments on multiple public datasets and performs comparative analysis with the ChatUnitTest [42] and EvoSuite [61] tools. In addition, ablation studies were carried out on static analysis, CoT prompting, and prompt templates to evaluate the contributions of each key component in improving the executability, logical completeness, and coverage of the generated test cases.

A. EXPERIMENTAL ENVIRONMENT

The hardware and software configuration of the experimental environment is as follows:

1) HARDWARE ENVIRONMENT

- ① CPU: AMD Ryzen 7 2700X Eight-Core Processor, 3.70 GHz, 8 cores and 16 threads
- ② Memory: 64 GB DDR4, 1432.2 MHz
- ③ Storage: 447 GB SSD (WD Green 2.5 480GB), 1.82 TB SSD (WDC WDS200T2G0A-00JH30)
- ④ Graphics Card: NVIDIA GeForce GTX 1060 3GB (3 GB VRAM)

```
1 package java_code_test;
2
3 *import static org.junit.jupiter.api.Assertions.*;
7
8 public class DiscountCalculatorTest {
9
10 @Test
11 public void testCalculateDiscountForNegativeAmount() {
12     DiscountCalculator calculator = new DiscountCalculator();
13     IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> {
14         calculator.calculateDiscount(-100);
15     });
16     assertEquals("Amount cannot be negative", exception.getMessage());
17 }
18
19 @Test
20 public void testCalculateDiscountForAmountGreaterThanOrEqualTo1000() {
21     DiscountCalculator calculator = new DiscountCalculator();
22     double result = calculator.calculateDiscount(1000);
23     assertEquals(200.0, result, 0.0001);
24 }
25
26 @Test
27 public void testCalculateDiscountForAmountBetween500And999() {
28     DiscountCalculator calculator = new DiscountCalculator();
29     double result = calculator.calculateDiscount(750);
30     assertEquals(75.0, result, 0.0001);
31 }
32
33 @Test
34 public void testCalculateDiscountForAmountBetween0And499() {
35     DiscountCalculator calculator = new DiscountCalculator();
36     double result = calculator.calculateDiscount(499);
37     assertEquals(24.95, result, 0.0001);
38 }
39
40 @Test
41 public void testCalculateDiscountForZeroAmount() {
42     DiscountCalculator calculator = new DiscountCalculator();
43     double result = calculator.calculateDiscount(0);
44     assertEquals(0.0, result, 0.0001);
45 }
46 }
```

FIGURE 5. Outputs of the test case generation and post-processing stages for statement coverage.

IllustrativeExampleMaven

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxt	Missed Lines	Missed Methods	Missed Classes
java_code		100%		100%	0	5	0	2
Total	0 of 32	100%	0 of 6	100%	0	5	0	2

FIGURE 6. Test case coverage report.

2) SOFTWARE ENVIRONMENT

- ① Operating System (OS): Windows 10 Professional Edition
- ② Programming Language and Integrated Development Environment: Python 3.12 (Cursor 0.50.7)
- ③ Open-source Project Runtime Environment: Eclipse 2025-03 (4.35.0), Maven 3.9.9
- ④ Unit Testing Framework: JUnit 5
- ⑤ LLMs: OpenAI GPT-4o
- ⑥ Other Tools: JavaParser library for extracting project dependencies and method signatures; JaCoCo 0.8.8 for measuring test case coverage

B. EXPERIMENTAL DATA

In this experiment, three representative Java open-source projects were selected as datasets to evaluate the applicability and performance of the IntelliUnitGen framework across

different types of codebases. These projects are Commons-CLI [47](a library for parsing command-line arguments), Commons-CSV [48](a utility library for reading and writing CSV files) and Binance Connector [49] (a Java SDK for accessing the Binance cryptocurrency exchange API). Both Commons-CLI and Commons-CSV are widely used and long-maintained open-source projects, which may have been included in the training data of LLMs, making them suitable for evaluating the test generation capabilities of LLMs on familiar codebases. These projects cover a variety of common application scenarios, such as command-line parsing and file processing, enabling a comprehensive assessment of LLM-generated unit test cases in terms of code coverage, readability, accuracy, and repair mechanisms. As a result, they are frequently used as evaluation benchmarks in research on automated unit test generation. In contrast,

TABLE 1. Basic information of the open-source datasets used in the experiment.

Project name	Version /submission number	The number of Java classes	Project type	The LLMs training data may contain
Commons-CLI	1.5.0	44	Tool Magazine	YES
Commons-CSV	1.10.0	18	Tool Magazine	YES
Binance-Connector	3.3.0	64	Java JDK	NO

version 3.3.0 of Binance-Connector was submitted to GitHub in November 2024, whereas the training data collection for the large model used in this study, GPT-4o [16], was finalized in October 2023. As a result, this dataset was not included in the training data of GPT-4o. This characteristic makes Binance-Connector a valuable addition for evaluating the generalization ability and robustness of the testing framework on previously unseen codebases. In summary, Commons CLI and Commons CSV represent typical projects familiar to the model, while Binance-Connector represents a novel project that the model has never encountered. Together, these three projects form a comprehensive combination of “familiar codebases” and “unseen codebases,” enabling an effective assessment of IntelliUnitGen’s performance and adaptability across different application scenarios. The basic information of the three datasets is shown in Table 1.

C. EXPERIMENTAL EVALUATION INDICATORS

This study uses coverage, generalizability, compilability & executability to evaluate the performance of the IntelliUnitGen framework in the task of unit test case generation.

(1) Coverage

The coverage is calculated as follows [62]:

① Statement Coverage

$$\begin{aligned} \text{Statement Coverage} &= \frac{\text{Number of Executed Statements}}{\text{Total Number of Statements}} \times 100\% \quad (2) \end{aligned}$$

② Branch Coverage

$$\begin{aligned} \text{Branch Coverage} &= \frac{\text{Number of Executed Branches}}{\text{Total Number of Branches}} \times 100\% \quad (3) \end{aligned}$$

③ Condition Coverage

$$\begin{aligned} \text{Condition Coverage} &= \frac{\text{Number of Conditions Covered}}{\text{Total Number of Conditions}} \times 100\% \quad (4) \end{aligned}$$

④ Decision Coverage

$$\begin{aligned} \text{Decision Coverage} &= \frac{\text{Number of Decision Outcomes Executed}}{\text{Total Number of Possible Decision Outcomes}} \times 100\% \quad (5) \end{aligned}$$

⑤ Path Coverage

Eq (6), as shown at the bottom of the next page.

This study evaluates only statement coverage and branch coverage. Experiments on the remaining three types of coverage will be conducted in future work.

1) GENERALIZATION ABILITY

The applicability of the generalization evaluation method across different programming languages, datasets, or coverage types.

2) COMPILABILITY & EXECUTABILITY

Compilability requires that the generated test code conforms to the syntactic specifications of the target programming language and includes essential structural components such as class definitions, method signatures, and import statements, so that it can be successfully compiled using standard build tools (e.g., Maven [63], Gradle [64]). Executability, on the other hand, emphasizes that the test cases can be executed at runtime through testing frameworks (e.g., JUnit [55], PyTest [59]) without causing runtime exceptions or logical errors.

To more accurately assess the usability of the generated results in real-world engineering environments, this study introduces two novel evaluation metrics based on compilability & executability: **First-Pass Compilation Success Rate (FPCSR)** and **First-Pass Execution Success Rate (FPESR)**.

① **FPCSR**: This metric measures the proportion of initially generated test cases for a target method in a given project that can be compiled successfully without any manual modifications. It reflects the effectiveness of the test generation process in terms of syntactic correctness, dependency completeness, and import specifications.

② **FPESR**: This metric assesses the proportion of initially generated test cases for a target method that can be executed successfully—i.e., compiled without errors and run without exceptions—without any manual intervention. It evaluates not only compilability but also the logical correctness and runtime stability of the generated code.

The calculation formulas for the two metrics are as follows:

$$FPCSR = \frac{NFPCTC}{TNGTC} \times 100\% \quad (7)$$

$$FPESR = \frac{NFPRTC}{TNGTC} \times 100\% \quad (8)$$

e terms *NFPCTC*, *NFPRTC*, and *TNGTC* respectively denote the Number of First-Pass Compilable Test Cases (NFPCTC), the Number of First-Pass Runnable Test Cases (NFPRTC), and the Total Number of Generated Test Cases (TNGTC).

By combining FPCSR and FPESR, the evaluation comprehensively assesses the stability, robustness, and practicality of the test case generation framework from two complementary dimensions: static correctness (compilability) and dynamic correctness (executability).

D. EXPERIMENTAL METHOD

We conducted both comparative experiments and ablation studies. In all experiments, the underlying model used was GPT-4o, with the temperature parameter set to 0.2 to reduce generation randomness and enhance the determinism of the generated test cases.

1) COMPARATIVE EXPERIMENTS

We selected the following three frameworks for comparative experiments:

- (1) EvoSuite [61]: A well-known JUnit test case generation tool, serving as a representative of non-LLM-based test generation approaches;
- (2) ChatUniTest [42]: A representative framework that primarily relies on the intrinsic processing and reasoning capabilities of LLMs for test case generation;
- (3) IntelliUnitGen: The test case generation framework proposed in this study, which integrates static analysis with prompt-based learning.

We used three open-source projects, Commons-CLI [47], Commons-CSV [48] and Binance-Connector [49], as experimental datasets and generated test cases for statement coverage and branch coverage on the classes within them. We compared the test coverage achieved by IntelliUnitGen with that of EvoSuite and the ChatUniTest framework. Based on the test coverage reports generated by JaCoCo, we conducted a comprehensive evaluation of IntelliUnitGen's performance across different projects and coverage objectives.

2) ABLATION STUDY

To evaluate the independent contributions of static analysis, CoT prompting strategy, and prompt template design within the IntelliUnitGen framework, we designed three sets of ablation experiments. Specifically, we removed each of the key components individually to construct the following IntelliUnitGen variants: IntelliUnitGen-Rm-Stc-Anls (with static analysis removed), IntelliUnitGen-Rm-CoT (with CoT prompting strategy removed), and IntelliUnitGen-Rm-Pmt-Dsgn (with prompt template design strategy removed). The impact of these removals on the performance of test case generation was then observed. Given that the purpose of the ablation study is to verify the individual role of each component rather than to exhaustively evaluate across all coverage metrics, we performed the evaluation solely on the dimension of statement coverage. This single-measurement approach is sufficient to support analytical conclusions while

avoiding redundant evaluation and allowing us to focus on core influencing factors.

In the ablation experiment on static analysis, we generated test cases using both the “framework integrating static analysis and prompt learning” and the “framework relying solely on prompt learning,” and compared their FPCSR and FPESR. This comparison aims to evaluate the role of static analysis in enhancing code structural correctness and executability.

In the CoT ablation experiment, we compared the results of the “framework with CoT prompting” against those of the “single-step prompting” framework to analyze the contribution of step-by-step reasoning to improvements in test case logical completeness and coverage.

In the prompt template design ablation experiment, we designed two sets of prompts: one using the structured prompt templates from IntelliUnitGen, and the other using a basic prompt template without coverage-oriented design. By comparing their differences in FPCSR and FPESR, we assess the impact of structured prompt design on the quality of the generated test cases.

By comprehensively comparing the results of the three groups of experiments mentioned above, we are able to assess the significance of static analysis, the CoT strategy, and prompt design in the performance of unit test case generation.

E. EXPERIMENTAL RESULTS AND ANALYSIS

Based on the experimental methodology described above, this section presents a comparative analysis of the statement coverage and branch coverage achieved by each test case generation framework, and further evaluates the contribution of key modules to overall performance through ablation studies.

The experimental results for statement coverage and branch coverage are shown in Table 2.²

The experimental results of the ablation studies are presented in Table 3.

The results in Table 2 and Table 3 indicate that:

1) STATEMENT COVERAGE

① Average Coverage

IntelliUnitGen achieves an average statement coverage of 67.0% across the three projects, slightly higher than that of ChatUniTest (63.4%) and significantly outperforming EvoSuite (50.0%). These results indicate that the strategy

²The source code of the IntelliUnitGen framework, the generated test cases for the experimental projects, and the corresponding JaCoCo coverage reports have been published at <https://github.com/yangzy0115/IntelliUnitGen>

Path Coverage

$$= \frac{\text{Number of Unique Paths Executed at least Once During Testing}}{\text{Total Number of Program Paths}}$$

*100%

(6)

TABLE 2. Comparison of statement coverage and branch coverage between IntelliUnitGen and Other frameworks (Unit: %).

TEST CASE GENERATION FRAMEWORK		Evosuite	ChatUniTest	Ours
Commons-CLI [47]	LINE	91.8	70.9	77.8
	BRANCH	84.3	–	73.0
Commons-CSV [48]	LINE	28.3	73.3	63.2
	BRANCH	70.9	–	72.5
Binance-Connector [49]	LINE	20.8	49.2	58.7
	BRANCH	75.7	–	63.0
Average coverage rate	LINE	50.0	63.4	67.0
	BRANCH	83.7	–	72.4

combining static analysis with prompt-based learning is effective in improving test coverage.

② Common-CLI Dataset

EvoSuite performs best on this dataset, reaching a coverage of 91.8%, mainly because the CLI project has a relatively simple structure, strong method independence, and clear execution paths, making it easier for search-based tools to enumerate paths and achieve high coverage. Nevertheless, IntelliUnitGen still outperforms ChatUniTest significantly (77.8% vs. 70.9%), suggesting that LLMs assisted by static analysis exhibit stronger understanding and generation capabilities when dealing with moderately complex code.

③ Common-CSV Dataset

ChatUniTest achieves the best performance on this dataset (73.3%), followed by IntelliUnitGen (63.2%), while EvoSuite performs poorly (28.3%). The CSV dataset includes a large number of logic components that depend on specific input formats, complex object construction, and invocation sequences, which are difficult for traditional structured tools to handle and generate valid paths. Despite not being specifically fine-tuned, IntelliUnitGen still achieves relatively high coverage, demonstrating the auxiliary effect of static structural information in testing complex logic classes. However, there remains a gap compared to ChatUniTest, suggesting that future improvements can be made by further optimizing prompt strategies and context modeling mechanisms.

④ Binance-Connector Dataset

IntelliUnitGen significantly outperforms ChatUniTest (49.2%) and EvoSuite (20.8%) with a statement coverage of 58.7%. The Binance codebase involves extensive cross-method dependencies, asynchronous invocations, and interactions with external APIs, which limit the path exploration capabilities of EvoSuite's random search-based strategy, resulting in substantially lower coverage. Although ChatUniTest benefits from the semantic understanding capabilities of LLMs, it often fails to capture deep logical invocation chains due to the absence of static dependency information. In contrast, IntelliUnitGen accurately identifies inter-class dependencies and method invocation relationships through static analysis, transforming them into prompt inputs that effectively guide the model to generate test cases covering

a broader range of execution paths. Consequently, IntelliUnitGen demonstrates superior adaptability and coverage capability in this highly complex scenario.

2) BRANCH COVERAGE

① Average Coverage

IntelliUnitGen achieves an average branch coverage of 72.4% across the three projects, slightly lower than EvoSuite's 83.7%. EvoSuite demonstrates superiority in branch coverage primarily due to its evolutionary algorithm, which repeatedly searches for uncovered branches. However, this approach is often constrained by path reachability issues in scenarios involving complex input construction and cross-dependency invocations. Although IntelliUnitGen performs slightly less effectively on certain datasets in terms of branch coverage, it exhibits better cross-dataset stability, higher executability, and stronger adaptability to previously unseen codebases.

② Commons-CLI Dataset

On this dataset, EvoSuite achieves a significantly higher branch coverage rate (84.3%) compared to IntelliUnitGen (73.0%). This advantage is mainly attributed to the relatively simple structure of the CLI project, where conditional branches are densely distributed and interdependencies are minimal, allowing EvoSuite's genetic algorithm to quickly cover a large number of branch paths. In contrast, IntelliUnitGen struggles to leverage its strengths in semantic reasoning and context awareness in such highly structured and semantically straightforward projects.

③ Commons-CSV Dataset

In this dataset, IntelliUnitGen attains a slightly higher branch coverage rate (72.5%) than EvoSuite (70.9%), marking the only instance among the three datasets where IntelliUnitGen outperforms EvoSuite. This result stems from the CSV project's relatively modular code structure, where functional interfaces are centered around core tasks such as reading, parsing, formatting, and writing CSV data. The moderate complexity of conditional branches and the depth of dependency chains align well with LLMs' strengths in semantic inference and structural perception. By statically extracting table parsing logic, boundary conditions, and exception handling paths, and incorporating CoT prompts, IntelliUnitGen is able to generate test cases with high coverage and logical completeness. Furthermore, due to the high predictability of data-driven branches in the CSV project, EvoSuite's advantage in random path exploration is partially diminished, allowing IntelliUnitGen to surpass it in branch coverage.

④ Binance-Connector Dataset

On this dataset, a noticeable gap remains between EvoSuite (75.7%) and IntelliUnitGen (63.0%) in terms of branch coverage. The Binance project contains numerous cross-module calls and external API dependencies, making certain branch paths difficult to cover through EvoSuite's structured search due to restrictions from external states and complex dependencies. However, IntelliUnitGen

TABLE 3. Ablation study results of IntelliUnitGen.

Test Case Generation Framework	INTELLIUNITGEN	IntelliUnitGen-Rm-Static-Anls	IntelliUnitGen-Rm-CoT	IntelliUnitGen-Rm-Prompt-Dsgn
Commons-CLI [47]	NFPCTC	14	8	7
	NFPRTC	12	7	5
	TNGTC	23	23	23
	FPCSR (Unit: %)	60.7	34.8	30.4
	FPESR (Unit: %)	52.2	30.4	21.7
Commons-CSV [48]	NFPCTC	5	3	1
	NFPRTC	4	2	1
	TNGTC	9	9	9
	FPCSR (Unit: %)	55.6	33.3	11.1
	FPESR (Unit: %)	44.4	22.2	11.1
Binance-Connector [49]	NFPCTC	27	10	5
	NFPRTC	25	6	4
	TNGTC	48	48	48
	FPCSR (Unit: %)	56.2	20.8	10.4
	FPESR (Unit: %)	52.1	12.5	8.3

demonstrates certain advantages in generating test cases that cover cross-method dependencies and complex input conditions by leveraging the integration of static analysis and prompt learning. As a result, its branch coverage approaches that of EvoSuite on this dataset.

In the experiments, all datasets were successfully processed by IntelliUnitGen after preprocessing. Notably, the largest class contained 1007 lines of code before preprocessing (i.e., the CommandLine class in CommonCLI). The results demonstrate that IntelliUnitGen maintains stable generation and execution capabilities even in scenarios involving large-scale classes. This further validates the scalability and robustness of the framework when handling long class files.

3) ABLATION STUDIES

To assess the independent contributions of static analysis, the CoT strategy, and prompt template design within the IntelliUnitGen framework, three groups of ablation experiments were conducted on the Commons-CLI, Commons-CSV, and Binance-Connector datasets, respectively. The experimental results are presented in Table 3 and are comparatively analyzed using the FPCSR and FPESR metrics.

① Ablation Study on Static Analysis

After removing the static analysis module (IntelliUnitGen-Rm-Static-Anls), both FPCSR and FPESR exhibited significant declines across all three datasets: on the CLI dataset, FPCSR dropped from 60.7% to 34.8% (a decrease of 25.9 percentage points), and FPESR fell from 52.2% to 30.4% (a decrease of 21.8 percentage points); on the CSV dataset, FPCSR decreased from 55.6% to 33.3%, and FPESR from 44.4% to 22.2%; on the Binance dataset, FPCSR dropped from 56.2% to 20.8%, and FPESR from 45.2% to 12.5%.

This downward trend indicates that static analysis plays a vital role in enhancing structural correctness and

executability of the generated code. The impact is particularly pronounced in the Binance project, the absence of static analysis causes the model to fail in correctly generating object instantiation and cross-class method invocation logic, resulting in a significant drop in compilation and execution success rates. In contrast, the CLI project exhibits fewer inter-class dependencies, leading to a relatively smaller decline. These findings demonstrate that static analysis not only improves the structural completeness and contextual consistency of the generated code, but also substantially enhances the framework's executability and stability in real-world scenarios involving multi-class dependencies.

② Ablation Study on CoT Strategy

After removing the CoT strategy (IntelliUnitGen-Rm-CoT), both FPCSR and FPESR exhibited noticeable declines, though the magnitude was slightly less severe than that observed in the static analysis ablation: on the CLI dataset, FPCSR dropped from 60.7% to 30.4%, and FPESR from 52.2% to 21.7%; on the CSV dataset, FPCSR declined from 55.6% to 11.1%, and FPESR from 44.4% to 11.1%; on the Binance dataset, FPCSR fell from 56.2% to 10.4%, and FPESR from 45.2% to 8.3%.

These results demonstrate that the step-by-step reasoning strategy contributes significantly to the logical completeness and assertion validity of generated test cases. This effect is particularly evident on datasets like CSV and Binance, which contain complex conditions and dependencies—without CoT, the model's ability to generate high-quality, executable test cases is substantially impaired.

③ Ablation Study on Prompt Design

After removing the optimized prompt templates and directly employing generic prompts (IntelliUnitGen-Rm-Prompt-Dsgn), both FPCSR and FPESR showed a significant decline across all three datasets: for CLI, FPCSR dropped

from 60.7% to 34.8%, and FPESR from 52.2% to 30.4%; for CSV, FPCSR decreased from 55.6% to 33.3%, and FPESR from 44.4% to 11.1%; for Binance, FPCSR declined from 56.2% to 16.7%, and FPESR from 45.2% to 8.3%.

Compared with static analysis and CoT, prompt template optimization contributes more substantially to ensuring executability and reducing syntax errors. This effect is particularly pronounced in the CSV and Binance projects, where unoptimized prompts are more likely to generate structurally incomplete or syntactically invalid code.

4) OVERALL ANALYSIS

In the preceding sections, we conducted independent analyses for each experimental table. In this section, we present a comprehensive analysis by aggregating the experimental results across the three datasets. From a holistic perspective, we further explore the performance differences in test coverage among different frameworks, with a particular focus on the following question:

Question1: Why does EvoSuite achieve higher statement coverage than branch coverage on the CLI dataset, whereas it exhibits higher branch coverage than statement coverage on the CSV and BINANCE datasets?

Answer: To address this question, we analyze from two perspectives: the test generation mechanism and the characteristics of the projects. Firstly, EvoSuite employs a genetic algorithm-driven structural search, aiming to maximize structural coverage during test case generation. The CLI project features a relatively simple structure, with a large number of non-conditional statements and sequential logic, making statements easier to cover; however, the conditional branches are relatively sparse and scattered, resulting in slower improvement in branch coverage. In contrast, the CSV and BINANCE projects contain a greater number of conditional judgments, error-handling logic, and nested branches. During the search process, EvoSuite is more likely to cover these logical branches, which leads to higher branch coverage than statement coverage.

Question2: Why do LLM-based frameworks generally achieve higher statement coverage than EvoSuite, while EvoSuite outperforms LLM-based frameworks in branch coverage?

Answer: This phenomenon is primarily attributed to the differences in test generation mechanisms and the respective strengths of the two types of approaches. On one hand, LLM-driven frameworks (e.g., IntelliUnitGen and ChatUniTest) leverage a holistic understanding of code structure and semantics to generate test cases that cover a wide range of basic execution paths. As a result, they perform particularly well in covering non-conditional statements, sequential logic, and common API calls, thereby achieving higher overall statement coverage. On the other hand, EvoSuite's genetic algorithm-based search process explicitly optimizes the exploration of path conditions in its fitness function. It is designed to continuously identify and cover previously uncovered branch conditions, making it especially

effective in projects with a high density of conditional logic, exception handling, and nested branches. Consequently, EvoSuite typically outperforms LLM-based frameworks in terms of branch coverage. This disparity highlights the differing emphases of the two paradigms: LLM-based methods excel in breadth-oriented coverage (statements), while EvoSuite is stronger in depth-oriented exploration (branches). It also suggests that future work could benefit from integrating the semantic generation capabilities of LLMs with structural search strategies to jointly optimize both coverage metrics.

The experimental results demonstrate that IntelliUnitGen outperforms the baseline approaches in terms of test coverage, stability, applicability and correctness. In particular, it exhibits superior testing capability and code comprehension in scenarios involving complex logic and multi-path structures.

VII. DISCUSSION

A. ALIGNMENT BETWEEN RESULTS AND RESEARCH OBJECTIVES

This study aims to address three core challenges in the process of automatically generating unit test cases using LLMs:

- (1) The generated test cases often suffer from structural incompleteness, lack of assertions, or compilation failures, which undermine their usability and correctness;
- (2) The absence of awareness regarding source code structure and dependency relationships leads to insufficient test coverage;
- (3) Prompt engineering lacks systematic design and controllability, making it difficult to adapt to complex code scenarios.

This study proposes the IntelliUnitGen framework, which integrates static analysis with prompt learning and incorporates the CoT strategy, aiming to address the aforementioned challenges and enhance the accuracy, coverage, and executability of generated test cases. The effectiveness of the proposed approach in supporting the research objectives has been validated through a series of experiments in the following aspects:

1) OBJECTIVE 1: ENHANCING THE STRUCTURAL COMPLETENESS AND COMPILABILITY OF TEST CASES

In the ablation studies, we introduced two evaluation metrics—FPCSR and FPESR—and systematically assessed the effectiveness of the framework in improving structural completeness and compilability through three sets of ablation experiments focused on static analysis, CoT reasoning strategy, and prompt design. The results demonstrate that the version incorporating static analysis significantly outperforms the baseline method based solely on prompt learning in both FPCSR and FPESR. This validates that static analysis can provide the model with precise class dependencies, field relationships, and control flow information, thereby enabling the generation of contextually consistent and dependency-complete test structures. The integration

of the CoT strategy further guides the model to construct test inputs and design assertions in a stepwise manner, significantly enhancing logical completeness. Moreover, the optimized prompt templates, which embed structured static information and reasoning cues, improve the model's stability and adaptability in generating test cases under complex scenarios. Taken together, the synergistic effect of static analysis, CoT reasoning, and prompt template optimization not only improves FPCSR and FPESR but also markedly enhances the structural and logical completeness of the test cases, highlighting IntelliUnitGen's advantages in stability and executability through multi-module collaboration.

2) OBJECTIVE 2: IMPROVING THE COVERAGE AND LOGICAL RIGOR OF TEST CASES

Across multiple open-source projects, IntelliUnitGen outperformed approaches that rely solely on LLMs in both statement and branch coverage. In projects familiar to GPT-4o, such as Commons-CLI and Commons-CSV, the results indicate that structured prompts can better activate the model's pretrained knowledge. Even in the previously unseen Binance-Connector project, IntelliUnitGen maintained a high level of coverage, demonstrating strong generalization capabilities.

3) OBJECTIVE 3: ENHANCING THE PRACTICALITY AND CONTROLLABILITY OF THE TEST GENERATION FRAMEWORK

The proposed framework supports modular static analysis and structured prompt design, enabling dynamic adjustment of prompt strategies according to different coverage objectives (e.g., statement, branch), thereby improving the controllability and flexibility of prompt construction. In addition, the incorporation of post-processing and manual repair mechanisms ensures the stability of the test generation process and its adaptability to engineering environments, thus enhancing the framework's deployment potential in real-world projects.

In summary, IntelliUnitGen effectively fulfills the predefined research objectives and demonstrates the feasibility and advantages of deeply integrating static analysis with prompt learning. It provides a structured and coverage-oriented solution for LLM-driven automated test generation approaches.

B. CHALLENGES AND LIMITATIONS

1) RESEARCH CHALLENGES

Although IntelliUnitGen has exhibited strong test generation capabilities and coverage performance across multiple datasets, it still faces several challenges in practical applications:

(1) Incompleteness in Static Analysis Information Extraction

The current static analysis approach primarily focuses on extracting class structures, method dependencies, and control/data flow paths, yet it does not fully account for complex language features such as generic type inference and

dynamic loading. This limitation may reduce the accuracy and completeness of the constructed prompts when dealing with framework-level code, interface-oriented programming, or systems with strong runtime behavior dependencies, thereby affecting the quality of the generated test cases.

(2) Rule-Driven Prompt Construction Strategy Lacks Adaptability

The current prompt generation process in IntelliUnitGen is based on predefined templates and synthesis rules derived from static information. Although this ensures strong controllability, it may require manual adjustment when applied across different projects or in complex code contexts. Furthermore, the granularity and ordering of prompt content significantly impact model output, and the lack of automatic optimization mechanisms makes it difficult to achieve optimal configurations for prompt engineering.

(3) Limited Generalization Capability of CoT Prompting Strategy

Although the CoT prompting strategy is effective in enhancing the understanding of complex logic, it still exhibits instability in scenarios such as test assertion construction and exception path coverage. Some generated test cases, while logically complete, fail assertions due to incorrect expected outputs. In addition, for classes with deep function call hierarchies and complex dependencies, the CoT reasoning chain is prone to failure or interruption, which degrades the overall quality of test generation.

2) LIMITATIONS

Although IntelliUnitGen has made significant progress in automated test case generation based on LLMs, it still exhibits certain limitations, which are primarily reflected in the following aspects:

(1) Certain requirements on users' professional background

Users of this framework are expected to possess fundamental knowledge of test coverage criteria and be capable of designing appropriate prompt templates. Therefore, the friendliness and usability of IntelliUnitGen for non-expert users still require further improvement.

(2) Lack of traceability and interpretability

During the test case generation process, when the model's output does not meet expectations or contains errors, it is often difficult to accurately trace the generation logic and identify the root causes of failure. For example, in certain extreme scenarios, the framework may produce test cases that fail to compile or execute successfully. In such cases, we are often limited to regenerating test cases using the large language model or manually correcting them, while it remains difficult to pinpoint and resolve the underlying issues within the LLMs itself. This black-box style generation mechanism leads to a lack of controllability in debugging and fine-tuning, thereby limiting the framework's usability and reliability in practical engineering applications.

(3) Dependence on External Model Performance and Response Stability

The current framework relies on invoking online LLMs (e.g., GPT-4o) for test case generation. However, the stability of model API services, invocation latency, and response consistency may lead to fluctuations in the generation results. Moreover, the behavior of large models may change with version updates, which poses additional challenges for the long-term maintenance of the framework.

(4) Runtime Performance Limitations

Although IntelliUnitGen can complete test case generation within acceptable time limits for small- to medium-scale open-source projects, the inclusion of static analysis and step-wise prompting (CoT) introduces additional computational overhead. When applied to industrial-scale projects containing a large number of classes and complex dependencies, the overall runtime efficiency may be adversely affected. This limitation restricts the framework's direct applicability in large-scale software engineering scenarios and highlights the need for future work to optimize the static analysis process and incorporate incremental generation mechanisms to enhance scalability.

(5) Context Window Limitation and Large File Handling

Currently, IntelliUnitGen's performance in handling large-scale classes or files with complex inter-class dependencies may be constrained by the context window length of LLMs, potentially leading to decreased coverage or executability. This limitation stems from the inherent challenges that existing LLMs face in modeling long-range contexts. Although the framework successfully processed the `CommandLine` class—consisting of up to 1007 lines prior to preprocessing—in our experiments, demonstrating the effectiveness of the preprocessing mechanism in large-class scenarios, its stability and efficiency remain to be further validated in even larger or structurally more intricate industrial-scale projects. To mitigate this issue, future work may explore the integration of retrieval-based methods, input chunking, or program slicing techniques to decompose large classes into manageable subunits. By leveraging static analysis information to maintain contextual coherence across chunks, these techniques can significantly enhance the framework's scalability and adaptability in industrial-scale applications.

C. THREATS TO VALIDITY

To ensure the credibility of the experimental results and the robustness of the research conclusions, we analyze potential threats to validity from the following four perspectives:

(1) Construct Validity

Construct validity concerns whether the experimental design accurately reflects the research objectives. In this study, we adopt statement coverage, branch coverage, FPCSR and FPESR as evaluation metrics, which can effectively measure the structural completeness and executability of the generated test cases. However, these metrics do not fully

capture other important dimensions such as assertion quality and semantic completeness, and thus may not comprehensively reflect the effectiveness of the test cases. Additionally, some evaluation metrics rely on the static analysis results of tools such as JaCoCo, whose accuracy may introduce a degree of uncertainty in the assessment.

(2) Internal Validity

Internal validity concerns whether the experimental results are affected by uncontrolled external factors. In this study, we strive to control variables such as prompt templates, invocation methods, and test execution environments to ensure consistency across comparative experiments. However, due to the inherent non-determinism of LLMs, even with identical inputs, the generated test cases may vary, thereby affecting the reproducibility of the results. To mitigate this issue, we conduct multiple runs and use averaged statistics, though some potential bias due to randomness may still remain.

(3) External Validity

External validity refers to the generalizability of experimental conclusions to other contexts. This study selects two open-source projects that are likely included in the LLM training corpus (Commons CLI [47] and Commons CSV [48]) and one project that is not covered by the training corpus (Binance-Connector [49]) to evaluate the framework's generalization capability, which provides a degree of representativeness. However, these three projects cannot fully represent all types of software systems and development scenarios. In particular, the applicability of the framework to industrial-scale complex systems and embedded systems requires further validation.

(4) Conclusion Validity

Conclusion validity concerns the consistency between the experimental results and the research conclusions. The experimental results in this study support the effectiveness of the combined strategy of static analysis and prompt learning. However, certain conclusions remain constrained by the sample size and the scope of experiments. To enhance the persuasiveness of the findings, future work may further expand the dataset scale, incorporate additional evaluation dimensions (such as assertion accuracy and defect detection capability), and adopt techniques like cross-validation to improve the reliability of the conclusions.

D. IMPLICATIONS FOR PRACTICE AND FUTURE RESEARCH

The proposed IntelliUnitGen framework has demonstrated promising results in both theoretical modeling and experimental validation, offering a novel approach and implementation pathway for automated unit test case generation. It provides the following practical implications and directions for future research:

1) IMPLICATIONS FOR SOFTWARE ENGINEERING PRACTICE

① **Promoting the Adoption of Automated Testing in Industrial Development:** IntelliUnitGen deeply integrates static analysis with prompt learning, offering a practical pathway for enterprises to incorporate high-quality unit testing

into Continuous Integration (CI) and Continuous Deployment (CD) pipelines. It is particularly well-suited for large-scale projects with stringent test coverage requirements.

② Enhancing Tester Efficiency and Coverage Capability: By statically extracting key contextual information from the source code to guide large language models in generating test cases, the framework significantly reduces the manual effort required for designing test data and assertions, thereby improving the efficiency and accuracy of test generation.

③ Improving Controllability and Stability of Test Generation: Compared with traditional black-box LLM-based generation strategies, IntelliUnitGen leverages structured prompts to guide test objectives and generation behavior. This helps address common issues in real-world projects, such as incomplete test logic and missing assertions.

2) IMPLICATIONS FOR FUTURE RESEARCH

① Systematic Optimization of Prompt Design Strategies: Future research should further investigate the universal structures, transferability, and semantic expressiveness of prompt templates, aiming to support more complex projects and diversified testing objectives.

② Automated Extraction and Semantic Abstraction of Static Analysis Information: Efficiently extracting and summarizing structural information that is useful for test generation across different programming languages and projects remains a key direction for enabling cross-language and cross-project testing systems.

③ Integration of Multi-Turn Reasoning and Feedback Mechanisms: Future work may explore how to incorporate prompt learning with automated feedback mechanisms—such as error log analysis and failure cause localization—to realize a closed-loop optimization process of test generation, verification, and repair.

④ Enhancing Capability in Complex Code Scenarios: When faced with intricate structures such as deep inheritance hierarchies, dynamic polymorphism, and concurrent logic, current models still exhibit limitations in understanding and generating stable test cases. This calls for the integration of fine-grained control flow modeling and advanced semantic analysis techniques.

⑤ Establishing Multi-Dimensional Evaluation Metrics and Benchmark Suites: Beyond coverage metrics, the evaluation of future test generation systems should also consider semantic completeness, assertion effectiveness, and debugging cost. Promoting the development of standardized benchmark suites will help both academia and industry better assess and compare the effectiveness of different approaches.

VIII. CONCLUSION

Existing unit test case generation techniques based on LLMs predominantly rely solely on the models' intrinsic understanding and generative capabilities. However, such approaches often suffer from limitations such as imprecise perception of code structure and lack of static contextual information, leading to test cases that are structurally

incomplete, lack necessary assertions, or exhibit insufficient coverage. To address these issues, this paper proposes a unit test case generation framework that integrates static analysis with prompt learning. The framework transmits statically analyzed class structures, dependency relations, method signatures, and other key information to LLMs, and guides the models to incrementally comprehend code structure, conditional logic, boundary values, and expected outputs. This enables LLMs to better interpret complex logical constructs and contextual semantic relationships, thereby achieving superior performance in terms of accuracy and coverage. The proposed approach offers a novel paradigm for leveraging LLMs in the domain of test case generation. It provides a new direction for controllable test case generation in complex code environments and establishes a solid foundation for enhancing the accuracy and practicality of automated unit testing. Furthermore, it delivers a more efficient, cost-effective, and controllable testing solution for various stakeholders involved in software projects.

IX. FUTURE WORK

In the future, we plan to further optimize and extend IntelliUnitGen, with the following key directions as our focus:

A. ENHANCING THE GENERALIZABILITY AND APPLICABILITY OF THE PROPOSED METHOD

This study primarily focuses on the generation of unit test cases. In the future, we plan to further abstract and elevate the strategy that combines static analysis with prompt learning, and propose a novel prompt learning technique—PromptW/StcAnls—specifically designed for source code processing tasks. This approach systematically maps multi-granularity structural information extracted through static analysis—such as code structure, inter-class dependencies, data flow paths, and conditional branch characteristics—into reusable prompt templates, and incorporates chain-of-thought reasoning to enhance the model's understanding of complex semantics. This technique can be applied not only to unit test case generation but also to a variety of tasks such as design pattern detection, code generation, and defect detection, thereby further improving the generalizability of our approach and expanding its application scope in the field of software engineering.

B. PROMOTING SEAMLESS INTEGRATION BETWEEN THE FRAMEWORK AND INTEGRATED DEVELOPMENT ENVIRONMENTS (IDES)

Although we have achieved full automation from test case generation to execution, the framework has not yet been integrated with IDEs, lacking deep integration with mainstream development toolchains. This limitation constrains its practicality and engineering deployment capabilities. In future work, we plan to advance the integration and practical application of the framework along two dimensions:

On the one hand, we plan to implement IntelliUnitGen v2.0 as a plugin to enable seamless integration

with mainstream IDEs such as Visual Studio Code, Visual Studio, Eclipse, IntelliJ IDEA, PyCharm, and the JetBrains Suite. This will allow users to generate, preview, and execute test cases with a single click directly within the development environment, thereby enhancing user interaction experience, lowering the entry barrier, and enabling efficient collaboration between development and testing. This improvement is expected to significantly enhance the efficiency of unit testing and provide a more intelligent and efficient solution for software unit testing.

On the other hand, to meet the demands of Continuous Integration (CI) and Continuous Deployment (CD) in modern software engineering, we plan to integrate IntelliUnitGen with mainstream CI/CD platforms such as GitHub Actions—similar to the one-click test generation approach provided by Diffblue Cover [65] in IDEs. By configuring triggers (e.g., code commits, pull requests, etc.), IntelliUnitGen can be automatically invoked to generate and execute test cases for newly added or modified modules, thereby enabling continuous and automated testing tasks. This functionality will support pipeline-based deployment, test result feedback, and log tracking, contributing to improved testing efficiency and enhanced code quality assurance in collaborative development environments.

C. EXPANDING THE COVERAGE TYPES AND EXPERIMENTAL DIMENSIONS

At present, this paper has only conducted comparative experiments on statement coverage and branch coverage using three datasets to evaluate IntelliUnitGen against ChatUniTest and EvoSuite, as well as ablation experiments comparing IntelliUnitGen with its variants—IntelliUnitGen-Rm-Stc-Anls (removing static analysis), IntelliUnitGen-Rm-Pmt-Dsgn (removing prompt design), and IntelliUnitGen-Rm-CoT (removing chain-of-thought reasoning)—using the FPCR and FPER metrics. However, the current experimental dimensions and datasets remain relatively limited. In future work, we plan to incorporate additional types of coverage experiments into the comparative experiments as well as extend the ablation studies to include factors such as context window size, static analysis granularity (e.g., method-level, class-level, project-level), and prompt strategy variants (e.g., zero-shot, few-shot prompts). Furthermore, we will introduce large-scale, multi-version, and multi-language open-source projects as new datasets to comprehensively verify the test case generation capability and adaptability of IntelliUnitGen under multi-dimensional criteria.

D. INTRODUCING LANGCHAIN TO ENABLE CHAINED REASONING FOR COMPLEX TEST GENERATION TASKS

Currently, IntelliUnitGen still suffers from insufficient contextual modeling when generating complex test cases. To address this limitation, we plan to incorporate the LangChain framework into IntelliUnitGen 2.0, decomposing the test case generation process into multiple subtasks

and guiding LLMs through a chained structure to perform multi-turn reasoning. This approach aims to enhance the model's ability to understand cross-method dependencies and complex logical structures. Inspired by the “generate–feedback–refine” closed-loop strategy employed in methods like Self-Refine [66], we will leverage LangChain's chaining mechanism to further refine the generation process into a series of interpretable subtasks, with feedback and optimization incorporated at each logical checkpoint, thereby achieving more fine-grained reasoning guidance and control. In addition, the modular architecture of LangChain will further improve the extensibility of the framework, enabling the flexible composition of task chains tailored to different testing requirements and supporting customizable test generation workflows. In addition, we will explore the integration of external knowledge sources (e.g., API documentation, project wikis, version histories) and dynamic feedback mechanisms to provide stronger support for functions such as test verification and assertion generation, and facilitating the evolution of IntelliUnitGen toward an intelligent testing agent.

E. EXPLORING SELF-REFINING PROMPT MECHANISMS

The current prompt design primarily relies on manually crafted prompt templates based on static analysis, lacking dynamic feedback and adaptive optimization capabilities. As a result, it is difficult to achieve automatic evolution of prompt content across different projects and testing objectives. To further enhance the intelligence and robustness of test case generation, we plan to introduce a Self-Refining Prompt mechanism in future work, aiming to construct a sustainable learning and self-iterative prompt generation strategy.

We intend to model the prompt optimization process as a feedback-driven closed-loop system: performance feedback—such as syntax errors, assertion failures, or insufficient coverage—will be collected during test case generation, compilation, and execution stages. Based on this feedback, the original prompt content will be automatically revised, rewritten, or supplemented to incrementally improve the prompt's effectiveness in guiding the model toward the testing objectives. We also plan to draw inspiration from the evolutionary self-optimization mechanism in Prompt-breeder [67], and integrate recent research on using LLMs as prompt optimizers [68], to support automatic prompt iteration and tuning across various projects and testing goals. This process may further incorporate prompt chaining techniques and reinforcement learning mechanisms to enable optimal prompt content search.

Moreover, by leveraging the model's intermediate reasoning traces and multi-turn dialogue context, we plan to design a context-aware prompt template adaptation module. This module will dynamically adjust prompt content based on different code structures and testing requirements, thereby enhancing generalization and robustness. This research direction will advance test case generation from “manual prompt

engineering” to “model-driven prompt evolution,” strengthening the framework’s adaptability to complex logic and dynamic scenarios, and laying a technical foundation for constructing intelligent testing agents.

F. OPTIMIZING RUNTIME PERFORMANCE AND ENHANCING INDUSTRIAL-SCALE ADAPTABILITY

At present, IntelliUnitGen demonstrates satisfactory runtime efficiency on small- to medium-scale projects. However, in industrial-scale projects, as the volume of source code, dependency complexity, and test coverage requirements increase, its runtime performance still faces certain bottlenecks. In future work, we plan to optimize IntelliUnitGen from three key perspectives: first, by adopting incremental analysis and parallelization strategies in the static analysis phase to reduce computational complexity and avoid redundant processing; second, by introducing caching mechanisms and lightweight models during the test case generation and execution stages to shorten prompt processing and inference time; and third, by deploying IntelliUnitGen in cloud-based or distributed environments to enhance its scalability and robustness in large-scale industrial projects. These improvements will enable the framework to better meet the runtime efficiency and stability requirements of enterprise-level software engineering environments while maintaining high-quality test generation.

G. OPTIMIZATION FOR CONTEXT WINDOW LIMITATIONS AND LARGE FILE PROCESSING

Given the inherent constraints of current large language models (LLMs) in modeling long-range contexts, we plan to enhance IntelliUnitGen by incorporating strategies such as retrieval-based methods, input chunking, and program slicing to address the challenges of test generation for large-scale classes and files with complex dependencies. These techniques enable the decomposition of oversized classes into multiple manageable sub-units, while leveraging static analysis results to preserve contextual integrity and consistency across sub-units. This enhancement aims to improve the framework’s adaptability and robustness in industrial-scale projects. In addition, we will explore integrating dynamic context extension mechanisms with long-context models—such as those supporting up to 100k tokens—to further boost the performance of IntelliUnitGen when handling cross-class dependencies and complex logic structures. This line of research not only alleviates the limitations imposed by existing context window lengths but also lays a solid technical foundation for deploying our framework in real-world engineering environments.

ACKNOWLEDGMENT

Note: The source code of IntelliUnitGen, the generated test cases for the experimental projects, and the JaCoCo coverage reports are available at: <https://github.com/yangzy0115/IntelliUnitGen>

REFERENCES

- [1] S. Goericke, *The Future of Software Quality Assurance*. Cham, Switzerland: Springer, 2020.
- [2] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*. Hoboken, NJ, USA: Wiley, 2011.
- [3] H. Oummarou and F. El Mansour, “Automatic generation of unit test data for dynamically typed languages,” *Indonesian J. Comput. Sci.*, vol. 12, no. 5, pp. 1–10, Oct. 2023.
- [4] M. Bozga, J.-C. Fernandez, and L. Ghirvu, “Using static analysis to improve automatic test generation,” in *Proc. Int. Conf. Tools Algorithms Construct. Anal. Syst. (TACAS)*, vol. 4, 2003, pp. 142–152.
- [5] A. Kallinjal Joshy, X. Chen, B. Steenhoek, and W. Le, “Validating static warnings via testing code fragments,” in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2021, pp. 540–552.
- [6] P. Zhang, J. Yu, and S. Ji, “ADF-GA: Data flow criterion based test case generation for Ethereum smart contracts,” in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. Workshops*, New York, NY, USA, Jun. 2020, pp. 754–761.
- [7] T. Su, C. Zhang, Y. Yan, F. Ling-ling, G. Pu, Y. Liu, Z. Fu, and Z. Su, “Towards efficient data-flow test data generation,” in *Theories Programming Formal Methods: Essays Dedicated To Jifeng He Occasion His 80th Birthday*. Cham, Switzerland: Springer, 2023, pp. 257–293.
- [8] T. Su, K. Wu, W. Miao, G. Pu, J. He, Y. Chen, and Z. Su, “A survey on data-flow testing,” *ACM Comput. Surv.*, vol. 50, no. 1, pp. 1–35, Jan. 2018.
- [9] D. Babić, L. Martignoni, S. McCamant, and D. Song, “Statically-directed dynamic automated test generation,” in *Proc. Int. Symp. Softw. Test. Anal.*, Jul. 2011, pp. 12–22.
- [10] P. Derakhshanfar, X. Devroey, A. Panichella, A. Zaidman, and A. van Deursen, “Generating class-level integration tests using call site information,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2069–2087, Apr. 2023.
- [11] P. Godefroid, “Test generation using symbolic execution,” in *Proc. IARCS Annu. Conf. Found. Softw. Technol. Theor. Comput. Sci. (FSTTCS)*, Wadern, Germany: Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2012, pp. 24–33.
- [12] T. Uehara, “Exhaustive test-case generation using symbolic execution,” *Fujitsu Sci. Tech. J.*, vol. 52, no. 1, pp. 34–40, 2016.
- [13] M. Staats and C. Păsăreanu, “Parallel symbolic execution for structural test generation,” in *Proc. 19th Int. Symp. Softw. Test. Anal.*, Jul. 2010, pp. 183–194.
- [14] L. H. Pham, Q. L. Le, Q.-S. Phan, J. Sun, and S. Qin, “Enhancing symbolic execution of heap-based programs with separation logic for test input generation,” in *Proc. Int. Symp. Autom. Technol. Verification Anal. (ATVA)*, 2017, pp. 209–227.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. 30 (NeurIPS)*, vol. 30, Long Beach, CA, 2017, pp. 5998–6008.
- [16] A. Hurst et al., “GPT-4o system card,” 2024, *arXiv:2410.21276*.
- [17] OpenAI. (2023). *GPT-4 Technical Report*. Accessed: Jul. 12, 2023. [Online]. Available: <https://cdn.openai.com/papers/gpt-4.pdf>
- [18] OpenAI. (Nov. 30, 2022). *Introducing ChatGPT*. Accessed: Jun. 8, 2025. [Online]. Available: <https://openai.com/index/chatgpt/>
- [19] T. B. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [20] A. Radford, J. Wu, and R. Child, “Language models are unsupervised multitask learners,” *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [21] A. Radford, K. Narasimhan, and T. Salimans. (2018). *Improving Language Understanding by Generative Pre-Training*. OpenAI. Accessed: Jun. 8, 2025. [Online]. Available: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- [22] M. Chen et al., “Evaluating large language models trained on code,” 2021, *arXiv:2107.03374*.
- [23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
- [24] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” 2023, *arXiv:2307.09288*.
- [25] A. Grattafiori et al., “The Llama 3 herd of models,” 2024, *arXiv:2407.21783*.
- [26] B. Rozière et al., “Code Llama: Open foundation models for code,” 2023, *arXiv:2308.12950*.

- [27] X. Bi et al., “DeepSeek LLM: Scaling open-source language models with longtermism,” 2024, *arXiv:2401.02954*.
- [28] A. Liu et al., “DeepSeek-v2: A strong, economical, and efficient mixture-of-experts language model,” 2024, *arXiv:2405.04434*.
- [29] A. Liu et al., “DeepSeek-V3 technical report,” 2024, *arXiv:2412.19437*.
- [30] D. Guo et al., “DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning,” 2025, *arXiv:2501.12948*.
- [31] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, “DeepSeek-coder: When the large language model meets programming—The rise of code intelligence,” 2024, *arXiv:2401.14196*.
- [32] Q. Zhu et al., “DeepSeek-Coder-v2: Breaking the barrier of closed-source models in code intelligence,” 2024, *arXiv:2406.11931*.
- [33] DeepSeek. *DeepSeek-Coder*. Accessed: May 31, 2025. [Online]. Available: <https://github.com/deepseek-ai/DeepSeek-Coder>
- [34] R. Li et al., “StarCoder: May the source be with you!” 2023, *arXiv:2305.06161*.
- [35] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, “No more manual tests? Evaluating and improving ChatGPT for unit test generation,” 2023, *arXiv:2305.04207*.
- [36] K. Li and Y. Yuan, “Large language models as test case generators: Performance evaluation and enhancement,” 2024, *arXiv:2404.13340*.
- [37] K. Liu, Z. Chen, Y. Liu, J. M. Zhang, M. Harman, Y. Han, Y. Ma, Y. Dong, G. Li, and G. Huang, “LLM-powered test case generation for detecting bugs in plausible programs,” 2024, *arXiv:2404.10304*.
- [38] A. Lops, F. Narducci, A. Ragone, M. Trizio, and C. Bartolini, “A system for automated unit test generation using large language models and assessment of generated test suites,” in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation Workshops (ICSTW)*, Los Alamitos, CA, USA, Mar. 2025, pp. 29–36.
- [39] A. Sapozhnikov, M. Olsthoorn, A. Panichella, V. Kovalenko, and P. Derakhshanfar, “TestSpark: IntelliJ IDEA’s ultimate test generation companion,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng., Companion*, Apr. 2024, pp. 30–34.
- [40] H. Zhou. (2023). *Ant Group’s Next-Generation Test Case Generation Technology Based on AIGC*. AiDD Summit, Shanghai, China. Accessed: Jun. 8, 2025. [Online]. Available: <https://www.aidd.vip/myzc-bj2023>
- [41] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.
- [42] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, “ChatUniTest: A framework for LLM-based test generation,” in *Proc. Companion 32nd ACM Int. Conf. Found. Softw. Eng.*, Jul. 2024, pp. 572–576.
- [43] Y. Wu, X. Zhou, A. Humayun, M. A. Gulzar, and M. Kim, “Generating and understanding tests via path-aware symbolic execution with LLMs,” 2025, *arXiv:2506.19287*.
- [44] S. R. Chowdhury, G. Sridhara, A. K. Raghavan, J. Bose, S. Mazumdar, H. Singh, S. B. Sugumaran, and R. Britto, “Static program analysis guided LLM based unit test generation,” in *Proc. 8th Int. Conf. Data Sci. Manage. Data (12th ACM IKDD CODS 30th COMAD)*, Dec. 2024, pp. 279–283.
- [45] Z. Wang, K. Liu, G. Li, and Z. Jin, “HITS: High-coverage LLM-based unit test generation via method slicing,” in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 1258–1268.
- [46] J. Lee, X. Wang, D. Schuurmans, M. Bosma, E. H., Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
- [47] Apache Software Foundation. *Apache Commons CLI*. Accessed: May 12, 2024. [Online]. Available: <https://github.com/apache/commons-cli>
- [48] Apache Software Foundation. *Apache Commons CSV*. Accessed: May 21, 2025. [Online]. Available: <https://github.com/apache/commons-csv>
- [49] Apache Software Foundation. *Apache Commons BINANCE*. Accessed: Aug. 4, 2025. [Online]. Available: <https://github.com/binance/binance-connector-java>
- [50] L. Wang, Y. Yuan, and G. R. Wang, “Design pattern detection techniques in software: Current status, challenges, and prospects,” *J. Softw.*, vol. 36, no. 6, pp. 2643–2682, 2025.
- [51] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, “A survey on large language models for code generation,” 2024, *arXiv:2406.00515*.
- [52] D. M. Zapkus and A. Slotkienė, “Unit test generation using large language models: A systematic literature review,” in *Proc. Vilnius Univ. Open Ser.*, 2024, pp. 136–144.
- [53] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, “A systematic literature review on large language models for automated program repair,” 2024, *arXiv:2405.01466*.
- [54] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 9th ed., New York, NY, USA: McGraw-Hill, 2019.
- [55] JUnit Team. *JUnit 5*. Accessed: May 27, 2025. [Online]. Available: <https://github.com/junit-team/junit5>
- [56] TestNG Team. *TestNG*. Accessed: May 20, 2025. [Online]. Available: <https://github.com/testng-team/testng>
- [57] MochaJS. *Mocha*. Accessed: May 24, 2025. [Online]. Available: <https://github.com/mochajs/mocha>
- [58] S. Purcell. (2001). *PyUnit: The Python Unit Testing Framework*. SourceForge. Accessed: Jun. 9, 2025. [Online]. Available: <http://pyunit.sourceforge.net/>
- [59] pytest-dev. *Pytest*. Accessed: May 27, 2025. [Online]. Available: <https://github.com/pytest-dev/pytest>
- [60] The Go Authors. *Source Code of the Go Testing Package*. Accessed: Jun. 2, 2025. [Online]. Available: <https://github.com/golang/go/tree/master/src/testing>
- [61] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *Proc. 19th ACM SIGSOFT Symp. 13th Eur. Conf. Found. Softw. Eng.*, Sep. 2011, pp. 416–419.
- [62] V. Khorikov, *Unit Testing: Principles, Practices, and Patterns*. New York, NY, USA: Manning Publications, 2020.
- [63] Apache Software Foundation. *Apache Maven Project Website*. Accessed: Jun. 2, 2025. [Online]. Available: <https://maven.apache.org/>
- [64] Gradle, Inc. *Gradle Official Website*. Gradle, Inc. Accessed: Jun. 2, 2025. [Online]. Available: <https://gradle.org/>
- [65] Diffblue. *Diffblue: AI Autonomous Unit Test-Writing Tool for Java Code*. Accessed: Aug. 12, 2025. [Online]. Available: <https://en.wikipedia.org/wiki/Diffblue>
- [66] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 46534–46594.
- [67] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, “Promptbreeder: Self-referential self-improvement via prompt evolution,” 2023, *arXiv:2309.16797*.
- [68] R. Ma, X. Wang, X. Zhou, J. Li, N. Du, T. Gui, Q. Zhang, and X. Huang, “Are large language models good prompt optimizers?” 2024, *arXiv:2402.02101*.



ZHUOYING YANG was born in 2004. She is currently pursuing the bachelor’s degree. Her research interests include software engineering, machine learning, big data, and blockchain technology. She is a Student Member of China Computer Federation (CCF).



LEI WANG was born in 1988. He received the Ph.D. degree. He completed postdoctoral research. He is currently an Associate Professor, a Distinguished Associate Research Fellow, and a Master’s Supervisor. His research interests include software engineering, machine learning, big data, and blockchain technology. He is a Professional Member of CCF.

...